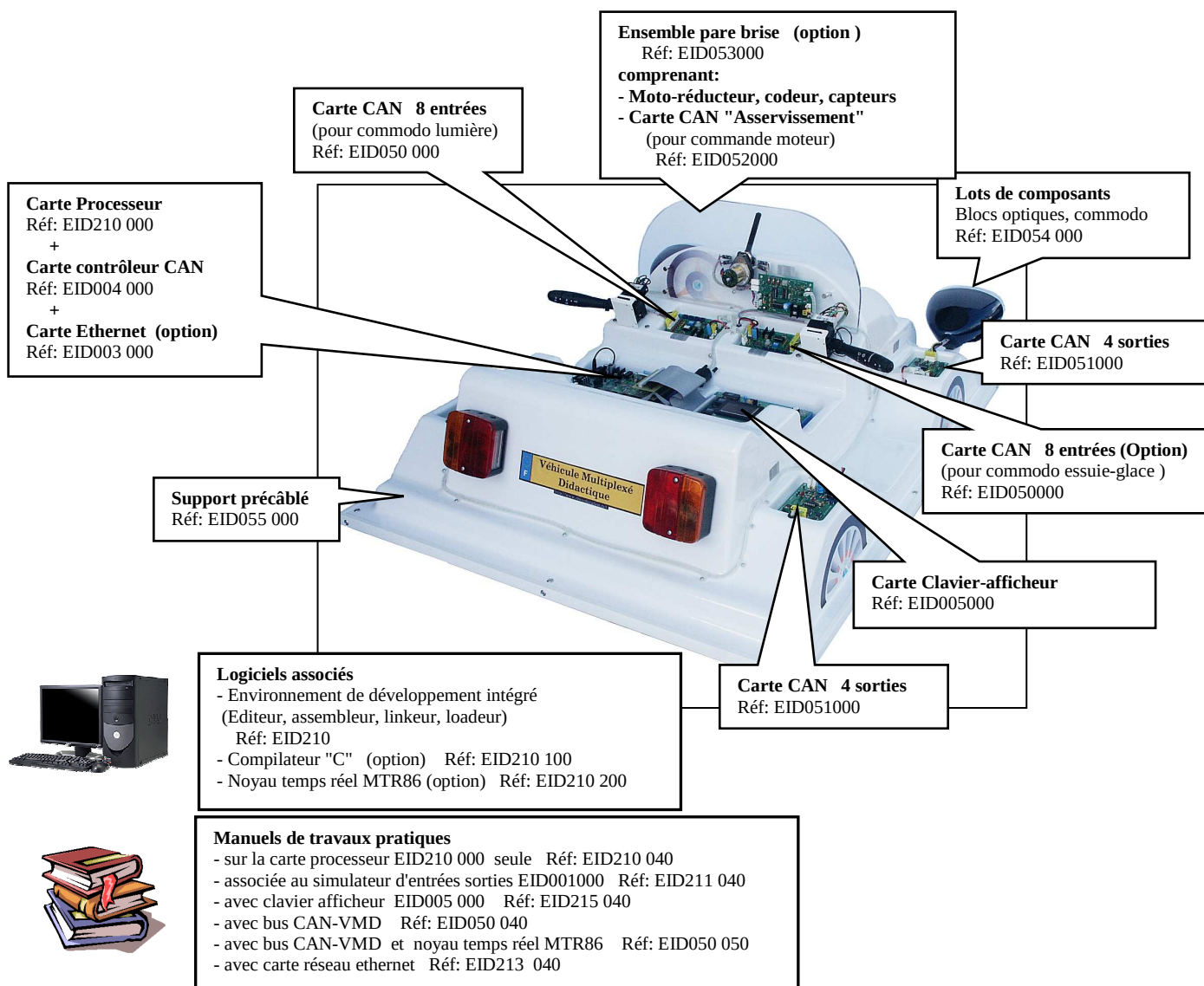


NOTICE TECHNIQUE

Systeme CAN - V.M.D. Véhicule Multiplexé Didactique



Version du : 14/07/2010

Référence document : EID055010

SOMMAIRE

1	PRESENTATION GENERALE	5
2	INSTALLATION ET CONFIGURATION	7
2.1	INSTALLATION MATERIEL	7
2.2	ELEMENTS DE CONFIGURATION PARTICULIERE DU LOGICIEL EID210	7
3	ELEMENTS DE TYPE "MATERIEL"	8
3.1	SCHEMA ORGANISATIONNEL D'ENSEMBLE	8
3.2	L'UNITE CENTRALE PROGRAMMABLE	8
3.3	LES MODULES D'INTERFACE CAN CONFIGURABLES	9
3.3.1	Les fonctions communes	9
3.3.2	Module 8 entrées logiques	12
3.3.3	Module 4 sorties de puissance	13
3.3.4	Module "Asservissement"	14
4	ELEMENTS DE TYPE "LOGICIEL"	17
4.1	TECHNIQUE DES ECHANGES PAR LA BUS CAN	17
4.1.1	Rappel de la structure d'une "trame"	17
4.1.2	Les différents types de trames	18
4.1.3	Définition des identificateurs	19
4.2	FICHIERS DE DEFINITION FOURNIS	22
4.3	STRUCTURES DE DONNEES SPECIFIQUES A L'APPLICATION VMD	22
4.4	BIBLIOTHEQUE DE FONCTIONS	24
5	ANNEXES	25
5.1	CARTES D'INTERFACE CAN	25
5.1.1	Plans d'implantation des composants	25
5.1.2	Schémas structurels	27
5.2	ELEMENTS SUR PARE BRISE	30
5.2.1	Moto-réducteur-codeur	30
5.2.2	Capteurs de fins de courses	31
5.3	FICHER DE DEFINITION CAN-VMD.H	32
5.4	FICHER DE DEFINITION PROPRE A LA CARTE ATON-CAN	35

1 PRESENTATION GENERALE

Le VMD (Véhicule Multiplexé Didactique) est un système didactique sur le RLI (Réseau locaux industriels).

Une séquence d'apprentissage élaborée autour de travaux pratiques sur un système didactique mettant en œuvre un réseau CAN est très riche d'enseignements :

- sur la connaissance des concepts généraux utilisés dans les réseaux locaux industriels (RLI), notamment le modèle O.S.I. (Open Systems Interconnection),
- sur les normes en vigueur,
- sur les termes usuels (voire normalisés) employés,
- sur la caractérisation d'une communication par RLI,
- sur les interfaces électriques et électroniques nécessaires,
- sur les protocoles de communication employés,
- sur la configuration des éléments configurables,
- sur la structuration des programmes les mettant en œuvre.

Le VMD utilise le bus CAN pour communiquer avec ses différents modules d'entrées et de sorties. Il a été développé en s'appuyant sur ce qui est développé dans l'automobile.

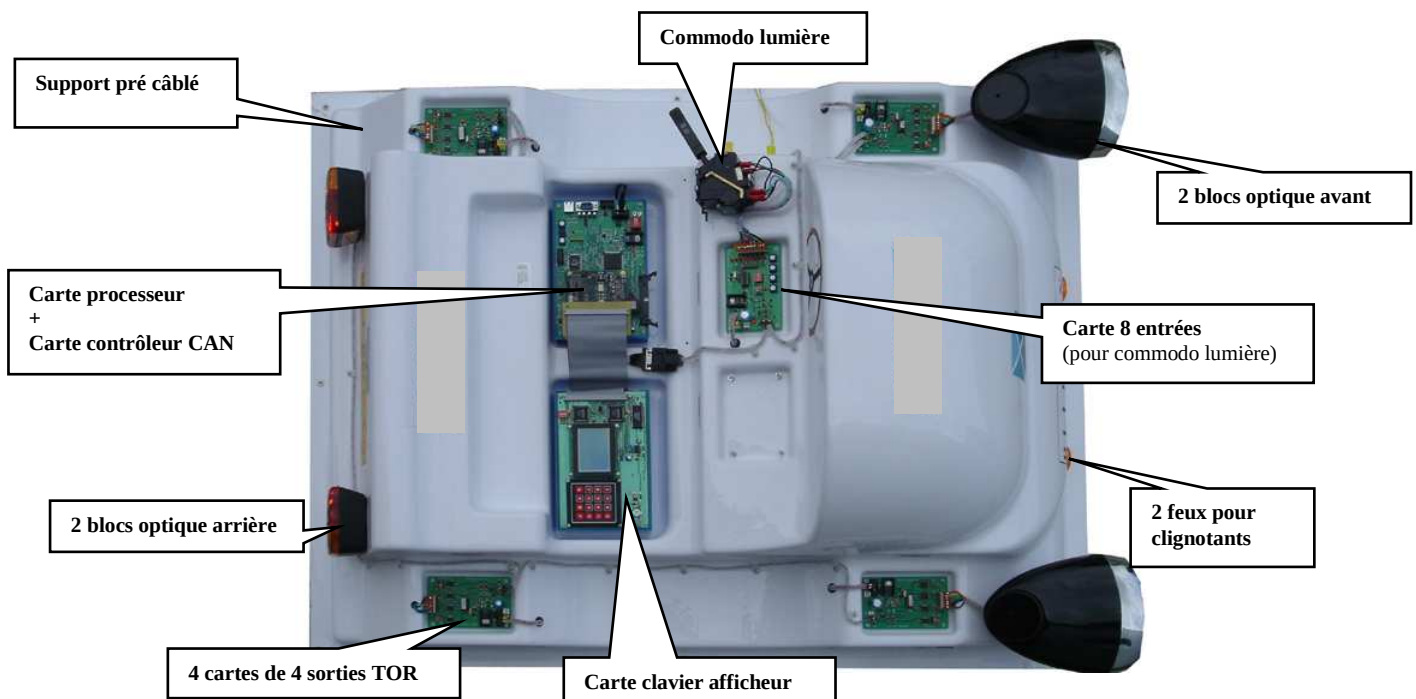
Dans le bus CAN, il n'y a que 2 couches normalisées dans le modèle OSI :

Couche physique (couche 1 OSI)

Couche liaison de donnée (couche 2 OSI).

Les autres couches ne font pas l'objet de normalisation dans le bus CAN.

Constitution de la version de base Réf: VMD01B



Constitution de la version complète Réf: VMD01C

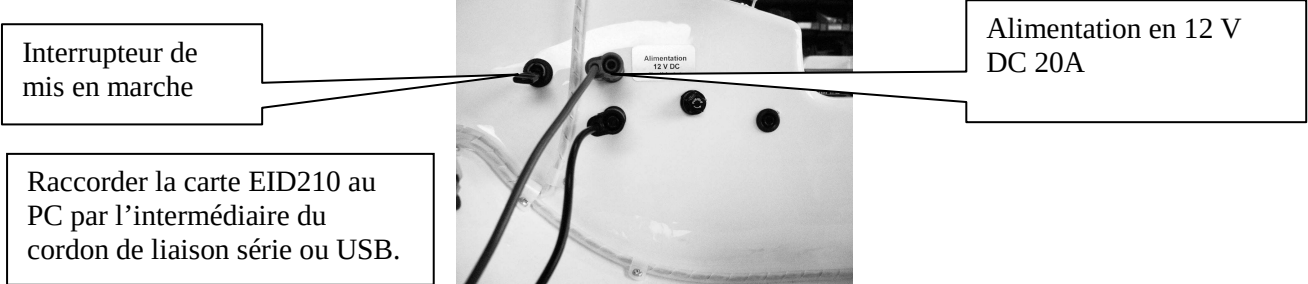
En plus des éléments présents sur la version de base (voir ci-avant), il est proposé les options suivantes:

- Pare-brise équipé (Moto-réducteur, capteur fins de courses, carte CAN "Asservissement" (pour commande du moteur par réseau CAN) Réf: EID053 000
- Carte CAN 8 entrées pour interfacier commodo essuie-glace Réf: EID051 000
- Carte réseau ethernet. Réf: EID003 000
- Noyau temps réel Réf : EID210 200

2 INSTALLATION ET CONFIGURATION

2.1 Installation matériel

Alimenter le VMD en +12 V DC



2.2 Eléments de configuration particulière du logiciel EID210

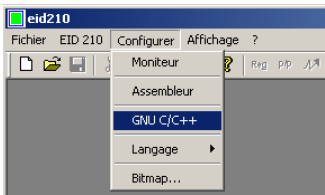
Pour installer l'environnement de développement EID210, exécuter le programme « SETUP.EXE » depuis le CD. (sous windows 2000 et XP professionnel ..., il faut avoir les droit suffisant pour pouvoir installer le logiciel).

Le logiciel EID210, qui comprend

- l'éditeur de texte (édition des programme source,
- le cross assembleur 68000,
- le cross compilateur c/c++ gnu C/C++

Il faut intégrer le fichier objet contenant les fonctions disponibles pour la gestion du réseau CAN.

Pour cela, faire "Configurer" puis "GNU C/C++"

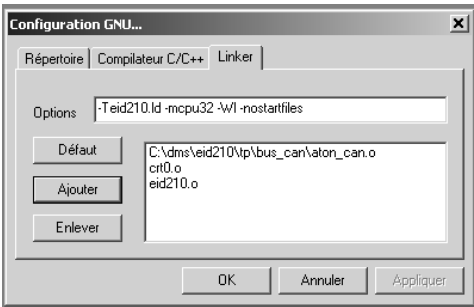


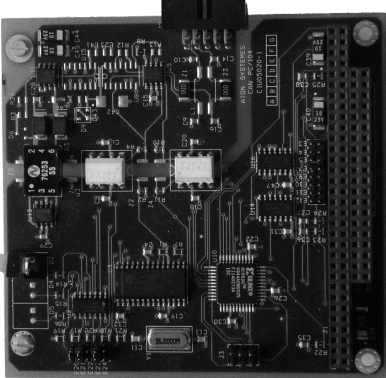
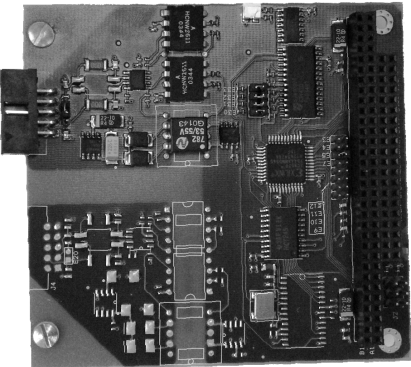
Dans la fenêtre qui s'ouvre alors, valider la fenêtre "Linker" et "Ajouter" le fichier "aton_can.o" se trouvant dans le répertoire "lib"

Dans la fenêtre "Options", donner éventuellement le chemin où se trouve le fichier eid210.ld qui se trouve également dans le répertoire "lib".

Remarque

Il existe 2 versions de carte contrôleur can ATON



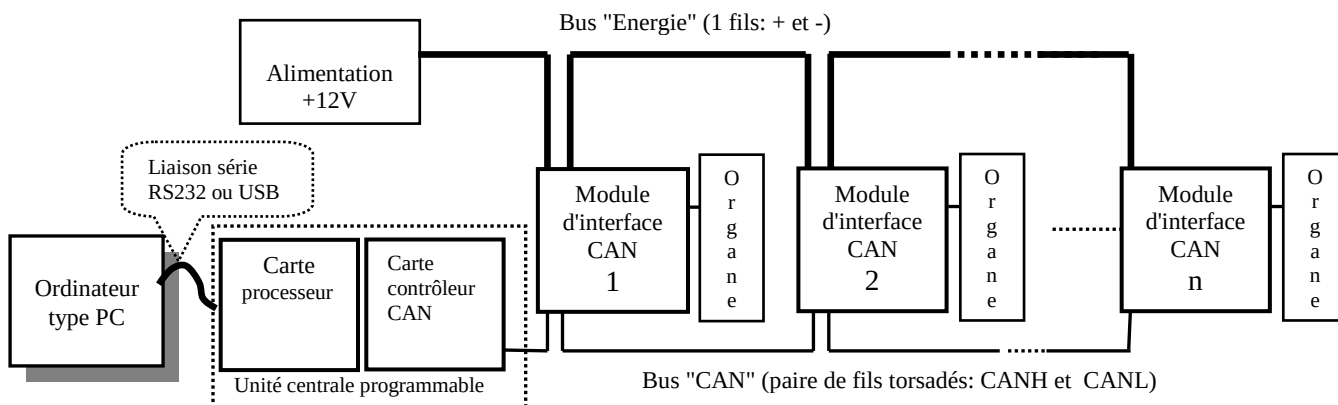
Version 1	Version 2
	
Aton_can.o Aton_can.h	Aton_can2.o Aton_can2.h

3 ELEMENTS DE TYPE "MATERIEL"

3.1 Schéma organisationnel d'ensemble

Le système comporte les éléments suivants:

- un ensemble "unité centrale" programmable en liaison avec un ordinateur de type PC,
- une source d'énergie (batterie 12V ou alimentation non autonome générant du 12V 20A,
- un certain nombre d'ensembles, chacun pouvant être constitué
 - * d'un module d'interface CAN configurable,
 - * d'un organe de type capteur, pré-actionneur voir actionneur, compatible avec le module associé,
- câbles de liaison.



Remarque :

- Des éléments de simulation d'entrées/sorties (LEDs, boutons poussoirs, commutateurs) ont été intégrés sur les modules d'interface CAN et permettent éventuellement de s'affranchir des organes réels.
- Dans sa version "VMD" (Véhicule Multiplexé Didactique) les organes reliés au module d'interface CAN peuvent être un commodo, un bloc optique avant, un bloc optique arrière, un moteur d'essuie glace ou de lève vitre, plafonnier ... etc.

3.2 L'unité centrale programmable

Elle comprend un certain nombre de cartes électroniques reliées entre elles par BUS parallèles :

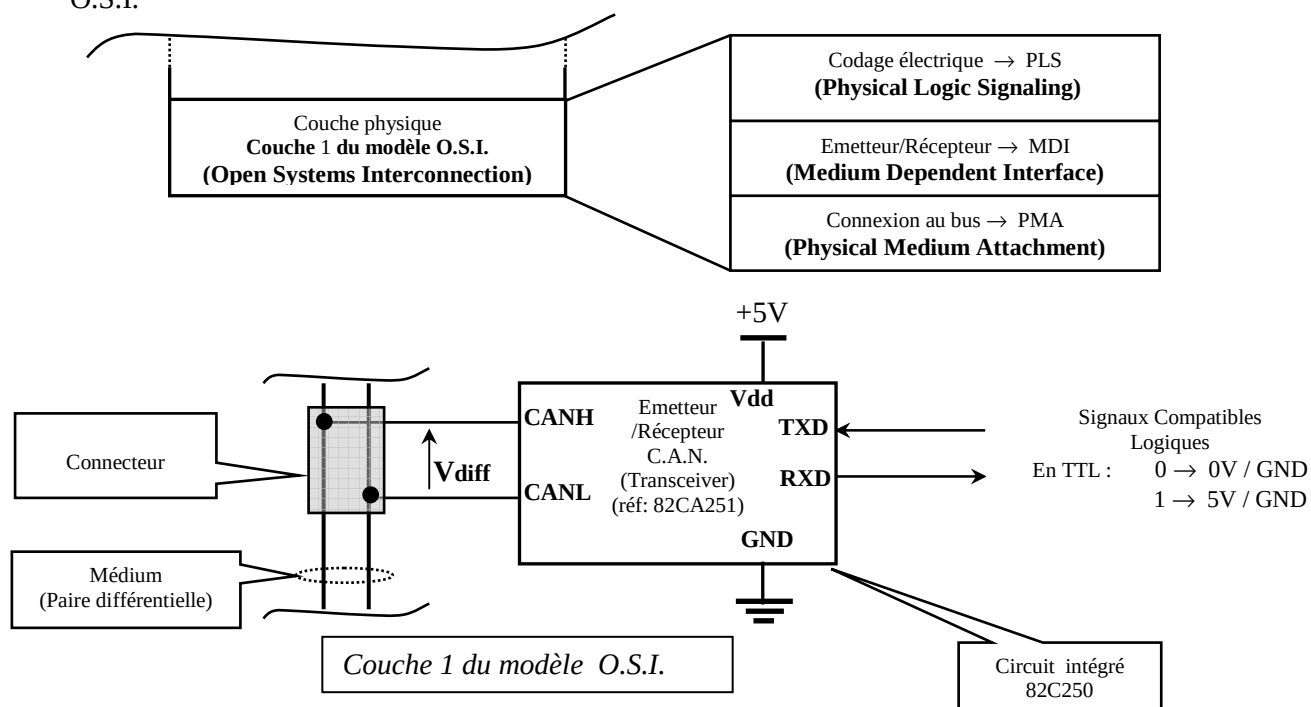
- cartes indispensables
 - carte processeur EID210 conçue autour du microprocesseur 32 bits Motorola 68332 (carte spécifique Didalab),
 - carte industrielle d'interface parallèle / CAN conçue autour du circuit SJA1000, au format normalisé "PC104" (carte produite par la société ATON SYSTEMES),
- cartes optionnelles
 - carte clavier 16 touches matricé et afficheur graphique (carte spécifique DIDALAB),
 - carte d'interface réseau "Ethernet" (carte spécifique DIDALAB),
 - toute carte au format PC104.

3.3 Les modules d'interface CAN configurables

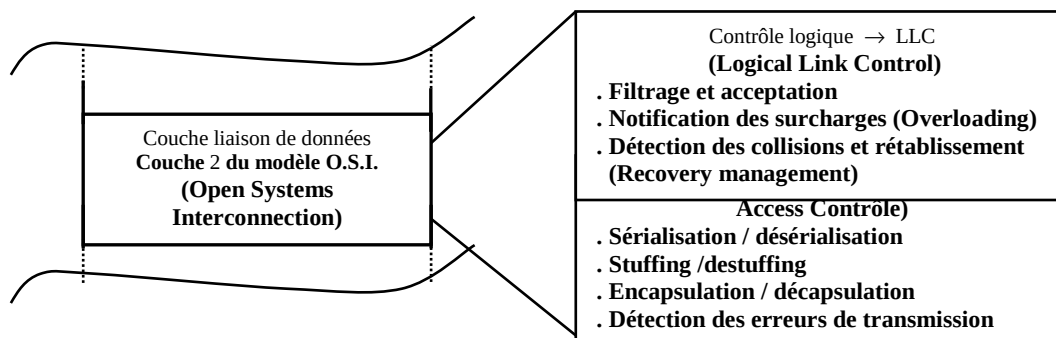
3.3.1 Les fonctions communes

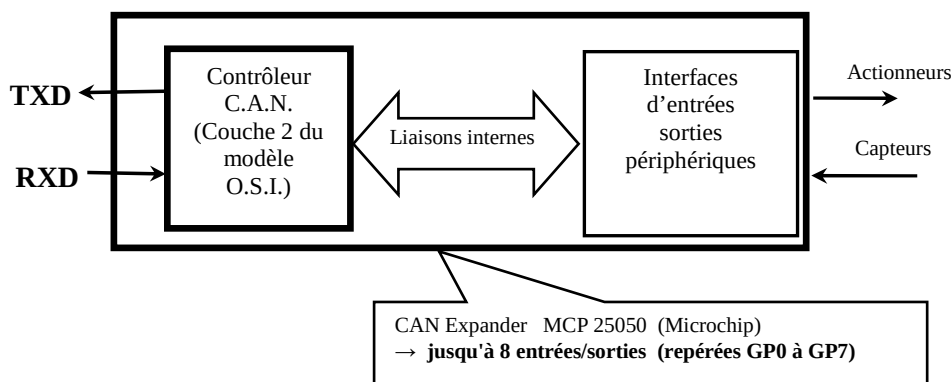
Tous ces modules comportent les éléments suivants :

- deux connecteurs de liaison au médium (fils de liaisons bus CAN) permettant de connecter les modules en série,
- deux connecteurs d'alimentation (bus "alimentation énergie"),
- un régulateur de tension générant la tension +5V nécessaire aux circuits intégrés inclus sur le module ainsi que les protections d'usage et une LED de visualisation de présence tension,
- une résistance de terminaison de bus connectable ou non par "jumper",
- un circuit intégré "émetteur récepteur de ligne" (réf:82CA251) réalisant l'interface électrique (mode différentiel côté bus CAN et mode référencé côté application), c'est à dire la couche 1 du modèle O.S.I.



- un circuit intégré d'extension d'entrées sorties (Réf:MPC25050) permettant la communication et l'interprétation des messages, c'est à dire la couche 2 du modèle O.S.I.





- un oscillateur à quartz nécessaire au circuit MPC25050,
- une LED de visualisation de communication en réception,
- une LED de visualisation de communication en émission.

Le composant "CAN Expander" MCP25050

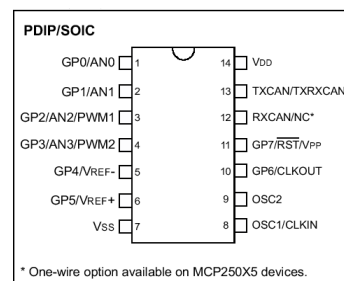
☞ Généralités

Pour plus de renseignements, se reporter à la "datasheet" du MCP25050 de Microchip

C'est un circuit intégré 14 broches disponible en version PDIP ou SOIC.

Côté interface CAN il satisfait la norme CAN V2.0B c'est à dire qu'il peut communiquer avec une vitesse de transmission qui peut atteindre 1Mbit/s. Certaines versions du circuit permettent une communication sur 1 fil (One-wire).

Côté application, il possède 8 lignes d'entrées/sorties (GP0 à GP7) configurables individuellement en entrée ou en sortie. Seule la ligne GP7 ne peut être utilisée en sortie.

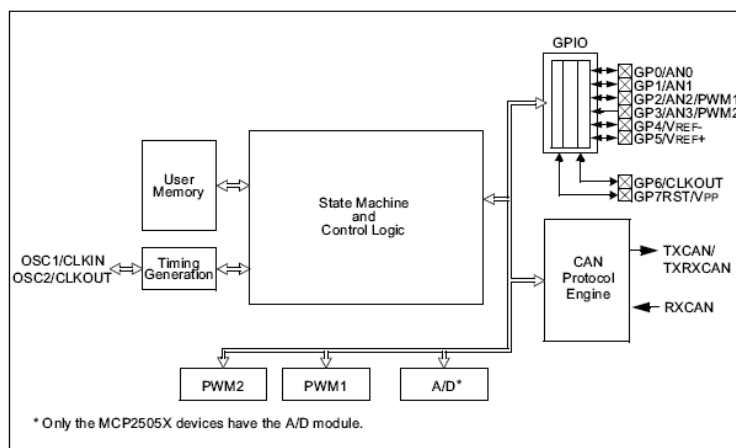


* One-wire option available on MCP250X5 devices.

Device	A/D	One-wire CAN
MCP25020	No	No
MCP25025	No	Yes
MCP25050	Yes	No
MCP25055	Yes	Yes

Deux liaisons (GP2 et GP3) peuvent être configurées en sorties modulées en largeur d'impulsions (PWM). Ces deux sorties peuvent être commandées indépendamment l'une de l'autre, fréquences et rapports cycliques sur 10 bits.

Quatre liaisons (GP0 à GP3) peuvent être des entrées analogiques. Les résultats numériques des conversions sont accessibles sur 10 bits.



* Only the MCP2505X devices have the A/D module.

Si on le souhaite, il est capable d'envoyer un message sans qu'il soit interrogé. Le message automatique peut avoir pour origine:

- changement d'état d'une entrée logique,
- dépassement d'un seuil d'une entrées analogique,
- base de temps périodique (fonction "scheduleur").

☛ La configuration de la vitesse de transmission

Cette vitesse dépend de la fréquence du signal d'horloge interne (imposée par le quartz) notée t_{osc} . Cette fréquence est divisée (passage dans un « perscaler ») pour obtenir la période t_Q (Time Quantum). Le coefficient de division BRP est choisi grâce à un mot de 6 bits BRP5 ... BRP0 (Ces bits font partie du registre de configuration n°1 CNF1 du circuit).

On obtient la valeur de t_Q grâce à l'expression :

$$t_Q = 2 * t_{osc} * (BRP + 1)$$

La durée de transmission d'un bit (Bit Time) est fonction de 3 paramètres :

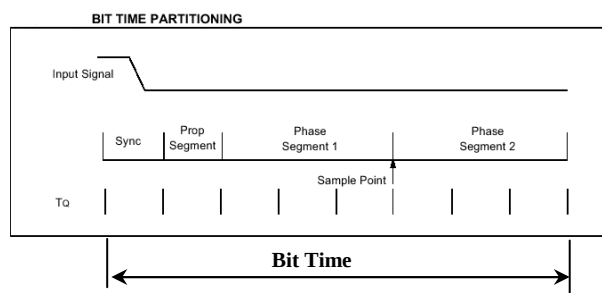
- un paramètre de synchronisation « SYN_Seg »
-> sur deux bits (SJW1 et SJW0) inclus dans le registre de configuration n°1
- un paramètre de propagation « Prop Segment »
-> sur 3 bits PRSEG2, PRSEG1 et PRSEG0 dans le registre de configuration n° 2
- un premier paramètre de phase « Phase Segment 1 »
-> sur 3 bits PHSEG12, PHSEG11 et PHSEG10 dans le registre de configuration n° 2
- un deuxième paramètre de phase « Phase Segment 2 »
-> sur 3 bits PHSEG22, PHSEG21 et PHSEG20 dans le registre de configuration n° 2

Les paramètres de phase permettent de définir l'instant d'échantillonnage (« Sample time»), instant où l'on prend la valeur logique du bus ce qui va donner l'état du bit.

On obtient la valeur de t_{BIT} (durée de transmission d'un bit) grâce à l'expression :

$$t_{BIT} = t_Q * (S_Seg + P_Seg + PH_Seg1 + PH_Seg2)$$

La vitesse de transmission est l'inverse de t_{BIT}



La répartition des paramètres dans les registres est donnée ci-contre.

(Voir « DATA SHEET » du circuit MCP25050 page 8 et 9)

Exemple de calcul de la vitesse de transmission:

La fréquence du quartz sur la carte est 16 Mhz

On a choisi BRP = 4

$$\rightarrow t_Q = 2 * (4 + 1) * t_{osc} = 10 / 16 \mu s = 0.625 \mu s$$

On a choisi :

SJW = 0 , PRSEG = 0

PHSEG1 = 8 et PHSEG2 = 8

D'après « DATA SHEET »

$$\rightarrow t_{BIT} = 16 * t_Q$$

En définitive :

$$t_{bit} = 16 * t_{sc1} = 16 * 0.625 \mu s = 10 \mu s$$

Soit une vitesse de $1 / t_{bit} = 100$ Kbit/s

CNF1 - CAN CONFIGURATION REGISTER 1

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
SJW1	SJW0	BRP5	BRP4	BRP3	BRP2	BRP1	BRP0
bit7 bit0							

CNF2 - CAN CONFIGURATION REGISTER 2

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
BTLMODE	SAM	PHSEG12	PHSEG11	PHSEG10	PRSEG2	PRSEG1	PRSEG0
bit7 bit0							

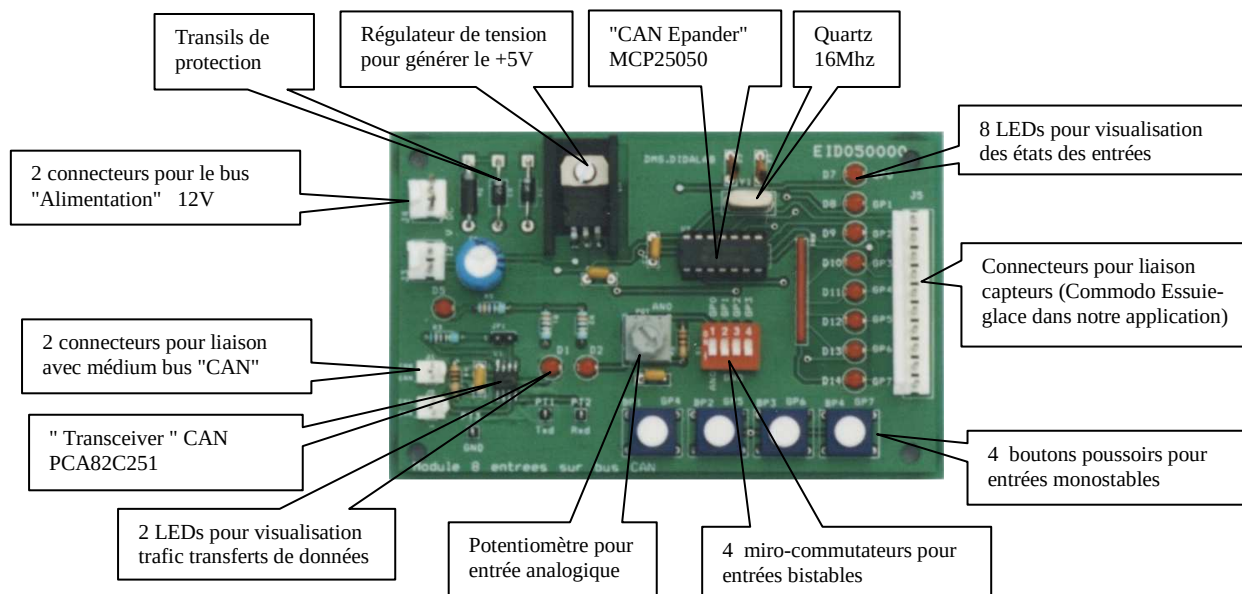
CNF3 - CAN CONFIGURATION REGISTER 3

U-0	R/W-0	U-0	U-0	U-0	R/W-0	R/W-0	R/W-0
---	WAKFIL	---	---	---	PHSEG22	PHSEG21	PHSEG20
bit7 bit0							

3.3.2 Module 8 entrées logiques

Le schéma structurel de cette carte est donné en Annexes.

Dans l'application objet, cette carte permet d'acquérir l'état du commodo "Essuie-glace"



Cette carte permet d'interfacer 8 entrées logiques "Tout Ou Rien", dont l'une peut être analogique.

En plus des éléments communs (présentés au chapitre précédent), cette carte comprend :

- 4 boutons poussoirs pour simuler des entrées de type monostables,
BP1 ↔ GP4 BP1 ↔ GP0 BP2 ↔ GP5 BP4 ↔ GP7
- 4 micro-commutateurs à 2 positions (fermé ou ouvert) pour simuler des entrées de type bistables,
S1-1 ↔ GP0 S1-2 ↔ GP1 S1-3 ↔ GP2 S1-4 ↔ GP3
- 1 connecteur 10 points permettant de relier le "commodo essuie-glace" (les 8entrées GP0 àGP7),
- 8 LEDs de visualisation des états des 8 entrées
- 1 potentiomètre pour entrée analogique.

Remarques :

- Si un capteur externe de type fin de course est relié au connecteur, le commutateur (ou bouton poussoir) correspondant doit resté en position "ouvert".
- Une LED s'allume si l'entrée correspondante est forcée à 0 (commutateur en position "fermé" ou bouton poussoir "appuyé").
- Ce module peut être utilisé comme module comportant des sorties logiques compatible "TTL". En effet le circuit MPC25050 est configurable. On peut donc, via le bus CAN, lui envoyer une trame qui définira si telle ou telle liaison est une entrée ou une sortie. On pourra donc envisager toute combinaison pourvu que la somme des entrées et des sorties ne dépasse pas 8.

Dans le cas d'une liaison configurée en sortie, il est impératif que le commutateur correspondant soit à l'état ouvert.

- Le potentiomètre n'est actif que si le commutateur "0" est fermé.

L'affectation des entrées sur la carte 8 entrées sur lequel est connecté le commodo Feux est la suivante :

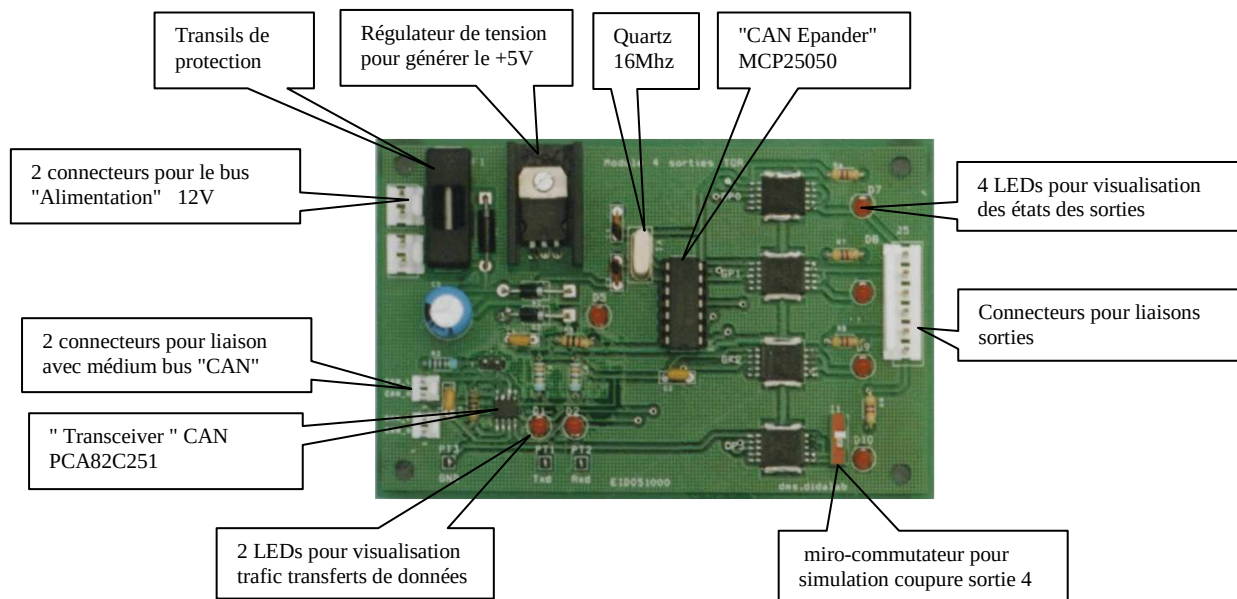
GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0
Entrée klaxon	Entrée stop	Entrée Clignotant droit	Entrée Clignotant gauche	Entrée Code	Entrée Phare	Entrée Warning	Entrée Veilleuse

- Pour définir, en phase d'initialisation, une liaison en "entrée", il faut écrire un '1' dans le bit correspondant du registre "GPDDR" (Data Direction Register).

3.3.3 Module 4 sorties de puissance

Le schéma structurel de cette carte est donné en Annexes.

Dans l'application objet, cette carte permet la commande de 4 sorties TOR de puissance



Ce module comporte:

- 4 interfaces de puissance permettant de piloter 4 charges électriques en "tout ou rien", sous 12V
- 4 Leds de visualisation des états des sorties puissance,
- 4 entrées de contrôle des charges,
- 1 entrée de simulation de coupure de charge.

Remarque :

- Le circuit intégré réalisant l'interface de puissance (Réf: VN05) génère un signal logique indiquant si une charge est connectée (contrôle du courant absorbé). Ces signaux logiques sont considérés comme entrée du système et permettent, dans le cas du VMD et pour la commande d'une ampoule, de contrôler le bon état de fonctionnement de celle-ci.
- Les sorties de puissance sont de type source, c'est à dire que les 4 charges auront pour point commun, la référence de potentiel (dans le cas d'un véhicule, le "-" de la batterie relié à la carcasce).
- Les circuits de puissance "VN05" peuvent accepter des charges électriques consommant jusqu'à 12 A en continu. Ils sont protégés contre les court-circuits ainsi que contre les dépassements de température.

L'affectation des entrées/sorties sur les cartes 4 sorties de puissance est la suivante :

GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0
Entrée Etat sortie GP3	Entrée Etat clignotant	Entrée Etat code	Entrée Etat veilleuse	Sortie (klaxon)	Sortie clignotant	Sortie code	Sortie Veilleuse

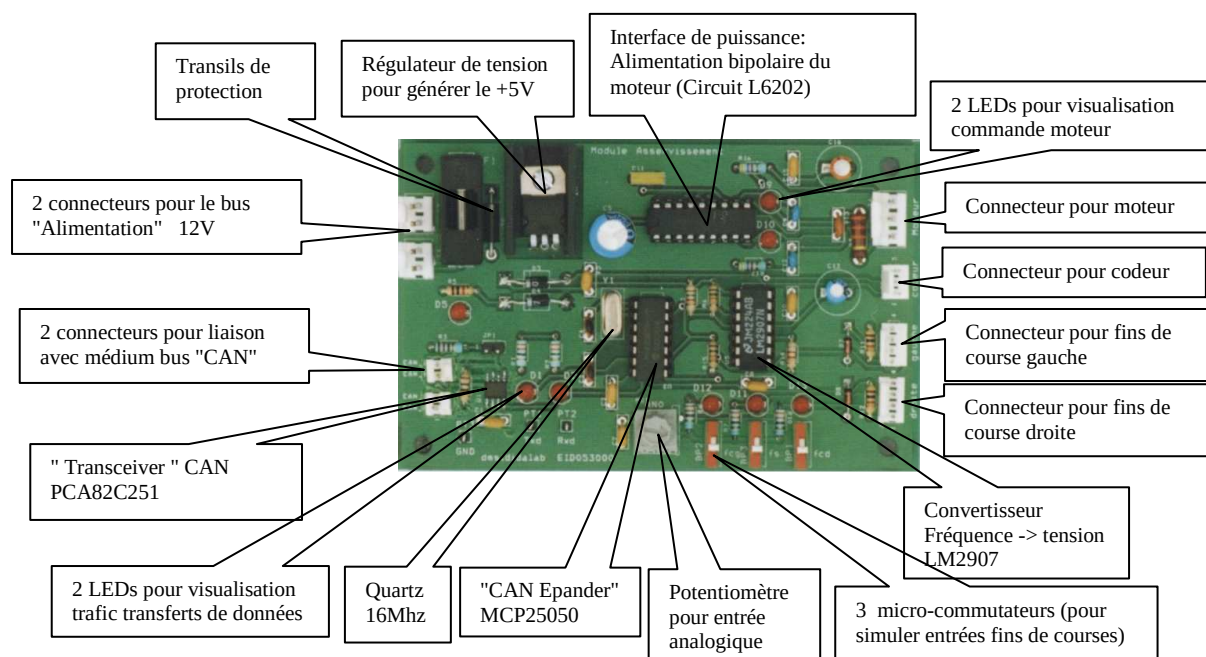
Remarques :

- La commande klaxon est active sur le module feux arrière gauche.
 - Pour définir, en phase d'initialisation, une liaison en "entrée", il faut écrire un '1' dans le bit correspondant du registre "GPDDR" (Data Direction Register) et pour définir, une liaison en "sortie", il faut écrire un '0' dans le bit correspondant du même registre "GPDDR".
- En définitive il faut écrire 11110000b ou F0h dans le "GPDDR".

3.3.4 Module "Asservissement"

Le schéma structurel de cette carte est donné en annexe.

Dans l'application objet, cette carte permet d'interfacer le moteur d'essuie-glace et d'acquérir les fins de courses ainsi qu'une image numérique de la vitesse du moteur (d'où la possibilité de commander le moteur en boucle fermée, d'où l'appellation "asservissement").



En plus des éléments communs (présentés au chapitre précédent), cette carte comprend:

- 1 interface de puissance permettant de piloter une charge électrique en "PWM" et dans les deux sens, sous 12V (destiné à alimenter un moteur à courant continu en bipolaire),
- 1 connecteur pour relier le moteur,
- 2 Leds de visualisation de l'état de la sortie puissance (l'une si commande >0 et l'autre si commande <0),
- 1 entrée potentiomètre analogique (générateur de consigne),
- 3 commutateurs de simulation de fins de courses (gauche, droit et sur-courses),
- 2 connecteurs pour fins de courses (gauche et droit, et sur-courses),
- 3 LEDs de visualisation des états des fins de course (droit, gauche et sur-courses)
- 1 convertisseur fréquence -> tension (destiné par exemple à une mesure vitesse à partir d'un capteur optique)
- 1 connecteur pour capteur optique de vitesse (ex: codeur incrémental 1 voie).

L'affectation des entrées/sorties et fonctions associées sur la carte "Asservissement" est la suivante :

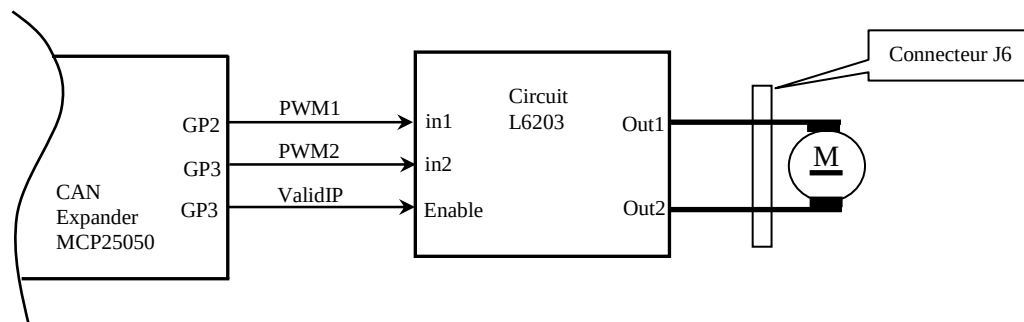
GP7	GP6	GP5	GP4	GP3	GP2	GP1	GP0
Entrée surcourse	Entrée Fin de course gauche	Entrée Fin de course droit	Entrée Validation Interface de puissance	Sortie PWM2	Sortie PWM1	Entrée Mesure vitesse	Entrée Tension potentiomètre interne

- Pour définir, en phase d'initialisation, une liaison en "entrée", il faut écrire un '1' dans le bit correspondant du registre "GPDDR" (Data Direction Register) et pour définir une liaison en "sortie", il faut écrire un '0' dans le bit correspondant du même registre "GPDDR".

En définitive il faut écrire 11100011b ou E3h dans le "GPDDR" du "CAN Expander".

Principe de l'interface d'alimentation moteur

L'élément de base de l'interface est le circuit intégré L6203 qui intègre une structure de puissance en H, ce qui permet une alimentation bipolaire du moteur à courant continu, en modulation de la largeur d'impulsions



- Le circuit intégré réalisant l'interface de puissance (L6202) peut permettre un courant de 4A en continu et 10A en impulsions non répétitif.
- Le signal "PW1" (signal GP2 du "CAN expander") permet d'imposer une tension positive aux bornes du moteur. Le signal "PW2" (signal GP3 du "CAN expander") permet d'imposer une tension négative. Ces signaux à rapports cycliques variables sont générés par le circuit MCP25050 à partir des ordres reçus via le BUS CAN.
- Le signal "ValidIP" (comme Validation de l'Interface de Puissance, signal GP4 du "CAN expander") permet (s'il est positionné à 1) de valider le circuit de puissance.

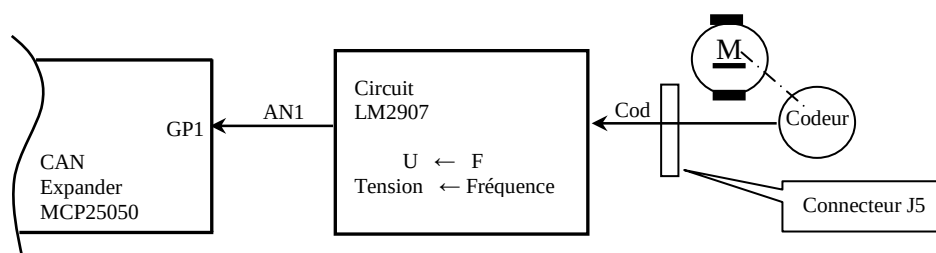
Remarque:

- Les liaisons GP2, GP3 et GP4 du "CAN Expander" devront être configurées en sorties.

Principe de la mesure de la vitesse du moteur

Un codeur incrémental 1 voie est accouplé à l'arbre moteur. Il délivre un signal repéré "COD" de fréquence proportionnelle à la vitesse de rotation.

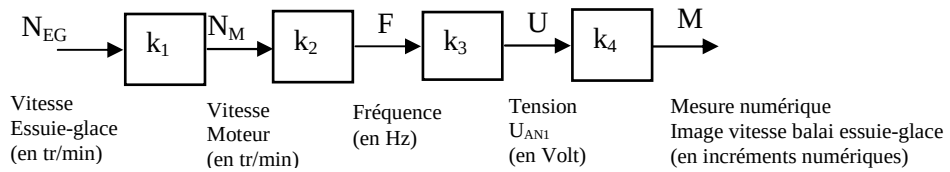
Le circuit LM2907 et les composants associés réalisent la conversion fréquence → tension. Le signal analogique repéré "AN1" est donc à un potentiel proportionnel à la fréquence, donc proportionnel à la vitesse de rotation.



Remarque:

La liaison GP1 du "CAN Expander" devra être configurée en entrée. Il faudra mettre en œuvre le convertisseur A→N intégré dans le "CAN Expander" pour avoir une image numérique de la vitesse de rotation du moteur (ou du balai d'essuie-glace).

Chaîne d'acquisition et de transfert de la vitesse du balai d'essuie-glace



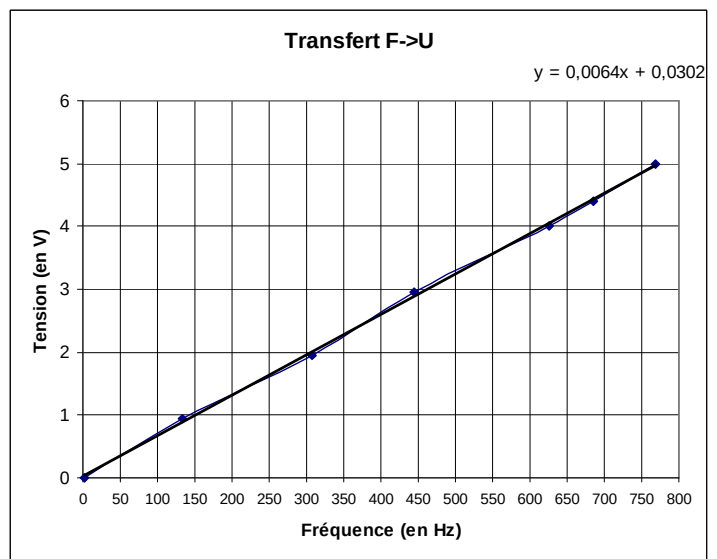
→ k_1 est l'inverse du coefficient de réduction du réducteur $k_1 = \frac{N_M}{N_{EG}} = 20$

→ k_2 est le coefficient de transfert dû au codeur incrémentale $k_2 = \frac{F}{N_M} = \frac{n_p}{60} = 0,2$

où $n_p = 12$ la résolution du codeur (nombre de périodes par tour codeur, donc par tour moteur)

→ k_3 est le coefficient de transfert du convertisseur fréquence → tension

Commande	Période	Fréquence	Tension
Moteur	T	F=1/T	U
0	1	1,00	0
50	0,0075	133,33	0,95
100	0,00325	307,69	1,95
150	0,00225	444,44	2,95
200	0,0016	625,00	4
220	0,00146	684,93	4,4
250	0,0013	769,23	5



On constate la linéarité de la caractéristique.

D'après l'équation de la courbe de tendance on en déduit la valeur du coefficient

$$k_3 = \frac{U}{F} = 0,0064 \text{ V/Hz}$$

→ k_4 est le coefficient de transfert du convertisseur tension → numérique

La conversion se fait sur 10 bits pour une plage de la tension d'entrée 0/5V

donc: $k_4 = \frac{1023}{5} = 204,6 \text{ inc/V}$

Soit un coefficient de transfert global:

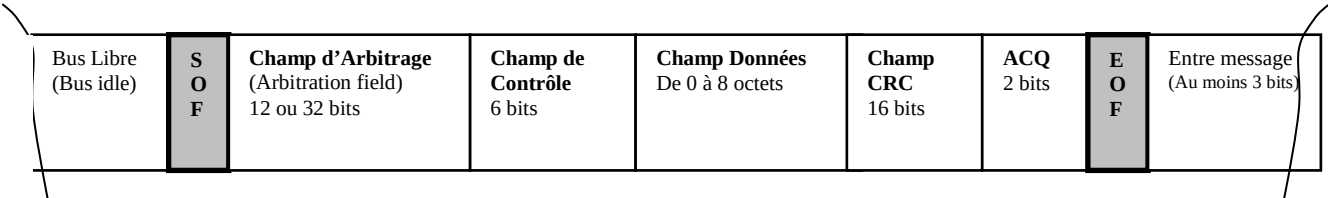
$$k = \frac{M}{N_{EG}} = k_1 \cdot k_2 \cdot k_3 \cdot k_4 = 5,24 \text{ inc/tr/min}$$

4 ELEMENTS DE TYPE "LOGICIEL"

4.1 Technique des échanges par la bus CAN

4.1.1 Rappel de la structure d'une "trame"

Une trame est une succession d'informations binaires circulant sous forme série:



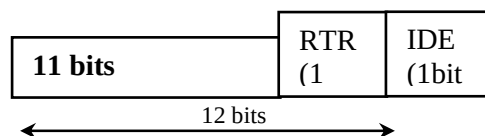
SOF Start Of Frame (Début de trame, sur 1 bit)

D'une durée d'un bit, son état est mis à 0 (état dominant) par la station qui prend l'initiative de débiter une transmission alors que le bus est libre.

Champ d'Arbitrage (12 bits en standard et ou 32 bits en étendu, cas du système étudié)

Le champ d'arbitrage est composé de l'identificateur et d'un bit permettant d'informer du type de trame (bit **RTR** Remote Transmission Request)

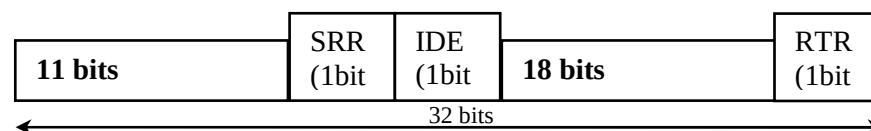
Format « standard » (version 2.0A du bus CAN):



RTR = 1 pour une trame de donnée (DATA frame)

RTR = 0 pour une trame d'interrogation (Remote frame)

Format « étendu » (Version 2.0B du bus CAN): C'est le format utilisé dans le système étudié



- Le bit **IDE** informe de la taille de l'identificateur :

→ est à 0 (dominant) en format standard

→ est à 1 (récessif) en format étendu.

Il est dans la zone de contrôle en mode standard et dans la zone d'identification en mode étendu.

- Le bit **SRR** (Substitute Remote Request) se substitue en lieu et place du bit RTR de format standard. Il est mis à 1. Ce bit est utile lorsqu'une trame au format étendu circule sur un bus CAN et rencontre une trame classique qui possède le même identificateur (partie haute).

Champ de Contrôle (sur 6 bits)

Ce champ comprend deux bits de réserve et 4 bits définissant la taille en nombre d'octets des données transmises (ce nombre est entre 0 et 8)

Champ de données

Réserve (sur 2 bits)

Longueur données
DLC (Data Length Code) (sur 4bits)

Les différentes données utiles transmises. Sa longueur dépend du nombre d'octets transmis défini dans le champ DLC.

Champ CRC (Cyclic Redundancy Code) sur 16 bits

Lors de l'émission, l'émetteur effectue un calcul (normalisé) sur les données à transmettre. Le résultat de ce calcul est transmis dans le champ CRC. Le récepteur effectue le même calcul et compare le résultat avec celui de l'émetteur transmis. Si une erreur est détectée, le récepteur renvoie un message d'erreur sous forme d'une trame interrogative.

Champ d'ACQuitement sur 2 bits (1 bit Slot suivi d'1 bit délimiteur)

L'émetteur envoie un bit à 1 (Récessif) et un récepteur ayant reçu correctement le message impose un zéro (dominant) : il acquitte donc le message.

EOF End Of Trame

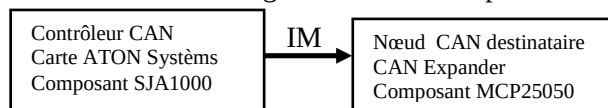
Chaque trame est terminée par 7 bits à 1 logique (récessif).

4.1.2 Les différents types de trames

L'utilisation du "CAN Expander" MCP25050 impose différents types de trames (d'après la page 2 dans la notice technique du "CAN Expander" MCP25050)

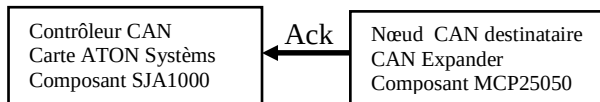
→ Trame "IM" (Input Message)

C'est en fait une trame de commande (ou missive) envoyée par le contrôleur à un nœud destinataire (ex: écriture d'une valeur dans un registre du "CAN Expander" MCP25050 destinataire)



→ Trame "Ack" (Acknowledge Message)

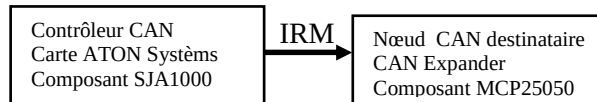
C'est une trame de réponse à une IM informant que le nœud destinataire s'est reconnu et a bien pris en compte la missive IM qui vient de circuler sur le bus.



→ Trame "IRM" (Information Request Message)

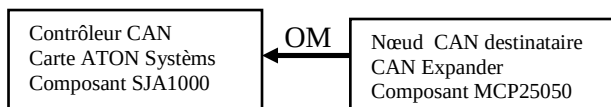
C'est en fait une trame de type requête (interrogation) envoyée par le contrôleur à un nœud destinataire lui demandant des informations

(ex: Lecture d'un ou de plusieurs registre(s) du "CAN Expander" MCP25050 destinataire)



→ Trame "OM" (Output Message)

C'est une trame de réponse à une IRM, envoyée par le nœud destinataire de l'IRM précédemment envoyée et contenant dans sa zone "Data" les informations demandées.



Conséquences:

- Dans un programme, suite à l'envoi d'une IM ou d'une IRM (utilisation de la fonction `Ecrire_Traine()`), il est impératif d'attendre la réponse (utilisation de la fonction `Lire_Traine()`), avant d'envoyer une nouvelle trame:

- cette réponse sera une "Ack" suite à une "IM"
- cette réponse sera une "OM" suite à une "IRM"

Lors du premier échange, il est conseillé de prévoir un "Time out" et l'affichage d'un message d'alerte dans le cas où il n'y a pas de réponse, afin d'éviter une situation bloquante.

Remarques:

- L'identificateur d'une "OM" est le même que celui de l'IRM qui est à l'origine.
- L'identificateur d'une "Ack" est différent que celui de l'IM qui est à l'origine.
- Deux particularités distinguent une trame d'interrogation (IRM) d'une trame d'information (IM):
 - le bit RTR est placé à 1 (dominant) alors qu'il est à 0 dans une trame de données,
 - la trame interrogative ne contient pas de données.

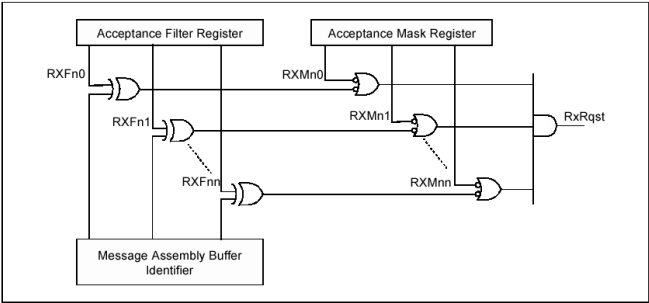
Trames particulières:

- A la mise sous tension un "CAN Expander MCP25050" signale sa présence par l'envoi automatique (sans interrogation) d'une trame de type "OB" (On Bus Message).
- Il est possible de configurer un "CAN Expander MCP25050" pour qu'il envoie automatiquement une trame
 - > lorsqu'une de ses entrées subit un changement d'état prédéfini (Pages 23 et 26 doc MCP25050)
 - > lorsqu'une entrée analogique prédéfinie croise un seuil de tension "Threshold detection" (Pages 23 et 35 doc MCP25050)
 - > à intervalle de temps régulier "Scheduled transmissions" (Pages 24 doc MCP25050)

4.1.3 Définition des identificateurs

☛ Généralités: La configuration des masques et des filtres "d'acceptance"

Un message circulant sur le bus ne sera pris en considération par le circuit que si l'identificateur associé au message a passé avec succès les obstacles du filtre et du masque.
Cette technique peut être déduite du schéma logique donné ci-après, ainsi que de la table de vérité.



FILTER/MASK TRUTH TABLE

Mask Bit n	Filter Bit n	Message Identifier bit n001	Accept or reject bit n
0	X	X	Accept
1	0	0	Accept
1	0	1	Reject
1	1	0	Reject
1	1	1	Accept

Note: X = don't care

En résumé, si on donne à un bit de masque l'état 1 logique un message ne sera accepté que si le bit correspondant de l'identificateur associé est du même état que celui donné au filtre.
Par contre, si on donne à un bit de masque l'état 0 logique, le bit correspondant de l'identificateur est masqué c'est à dire que son état n'est pas vérifié.
La définition des masques et des filtres s'effectue grâce à un certain nombre de registres destiné à cet effet (voir tableau ci-après).

Rappel :

Dans la version "standard" du bus CAN (version V2.0A) l'identificateur est sur 11 bits. Dans ce cas les bits de l'identificateur sont repérés **SID10 ... SID0**.
Dans la version "étendue" du bus CAN (version V2.0B) l'identificateur est sur 29 bits. Dans ce cas les bits de l'identificateur sont repérés **EID17 ... EID0** (ce sont les poids faibles de l'identificateur, les poids forts étant l'identificateur standard).

- Il y a deux séries de registres dans lesquels on charge les valeurs des masques et des filtres :
- la série d'indice 0 destinée aux trames interrogatives ("Information Request"),
 - la série d'indice 1 destinée aux entrées de données ("Input message").

Les registres définissant les masques

RXMSIDH - ACCEPTANCE FILTER MASK STANDARD IDENTIFIER HIGH							
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID10	SID9	SID8	SID7	SID6	SID5	SID4	SID3*
bit7							bit0

R = Readable bit
W = Writable bit
U = Unimplemented, read as '0'

RXMSIDL - ACCEPTANCE FILTER MASK STANDARD IDENTIFIER LOW							
R/W-x	R/W-x	R/W-x	U-0	R/W-x	U-0	R/W-x	R/W-x
SID2	SID1	SID0	—	EXIDE	—	EID17	EID16
bit7							bit0

RXMEID8 - ACCEPTANCE FILTER MASK EXTENDED IDENTIFIER MID							
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit7							bit0

RXMEID0 - ACCEPTANCE FILTER MASK EXTENDED IDENTIFIER LOW							
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0
bit7							bit0

D'après "DATA SHEET" du circuit pages 12, 13 et 14

Les registres définissant les filtres

RXFNSIDH - ACCEPTANCE FILTER N STANDARD IDENTIFIER HIGH							
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
SID10	SID9	SID8	SID7	SID6	SID5	SID4	SID3*
bit7							bit0

R = Readable bit
W = Writable bit
U = Unimplemented, read as '0'

RXFNSIDL - ACCEPTANCE FILTER N STANDARD IDENTIFIER LOW							
R/W-x	R/W-x	R/W-x	U-0	R/W-x	U-0	R/W-x	R/W-x
SID2	SID1	SID0	—	EXIDE	—	EID17	EID16
bit7							bit0

RXFNEID8 - ACCEPTANCE FILTER N EXTENDED IDENTIFIER MID							
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID15	EID14	EID13	EID12	EID11	EID10	EID9	EID8
bit7							bit0

RXFNEID0 - ACCEPTANCE FILTER N EXTENDED IDENTIFIER LOW							
R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
EID7	EID6	EID5	EID4	EID3	EID2	EID1	EID0
bit7							bit0

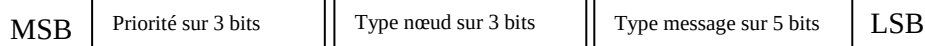
☛ Application au CAN-VMD

On utilise le mode d'identification "étendu", car il est indispensable si on souhaite adresser individuellement les différents registres du circuit d'interface MCP25050, circuit de base des modules d'interface CAN (voir notice technique du circuit MCP25050, page 22).

Rappel:

Dans le mode d'adressage le 0 (dominant) est prioritaire sur le 1 qui est (récessif) (voir documentation technique sur réseau CAN).

On envisage les champs d'adressage suivants pour les 11 bits de la partie "standard" de l'identificateur:



La priorité:

- 000 "Asservissement" (Priorité la plus haute)
- 001 "Commodo" (lumière ou essuie glace)
- 010 "Afficheur clavier"
- 011 "Feux"

Le type de Nœud:

- 001 Nœud "Asservissement"
- 010 Nœuds "Comodo lumière"
- 011 Nœuds "Comodo essuie glace"
- 100 Nœuds "Feux avant gauche"
- 101 Nœuds "Feux avant droit"
- 110 Nœuds "Feux arrière gauche"
- 111 Nœuds "Feux arrière droit"

Le type de message:

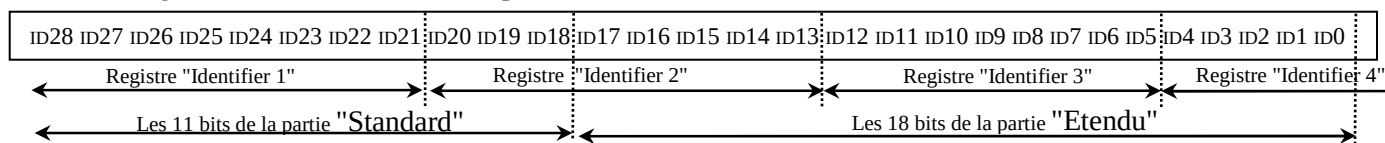
- 00001 -> repère RXF0 -> Réception d'une IRM (Information Request Message) et réponse OM (Output Message).
- 00010 -> repère RXF1 -> Pour écrire dans un registre (Input Message)
- 00100 -> repère TXID0 -> Pour On bus message (au démarrage pour informer présence nœud)
- 01000 -> repère TXID1 -> Pour acquittement suite à un IM (Input Message)
- 10000 -> repère TXID2 -> Pour messages automatiques (ex: conversions cyclique Analogique-Numérique)

Détermination des identificateurs, valeurs à charger dans les registres du SJA1000

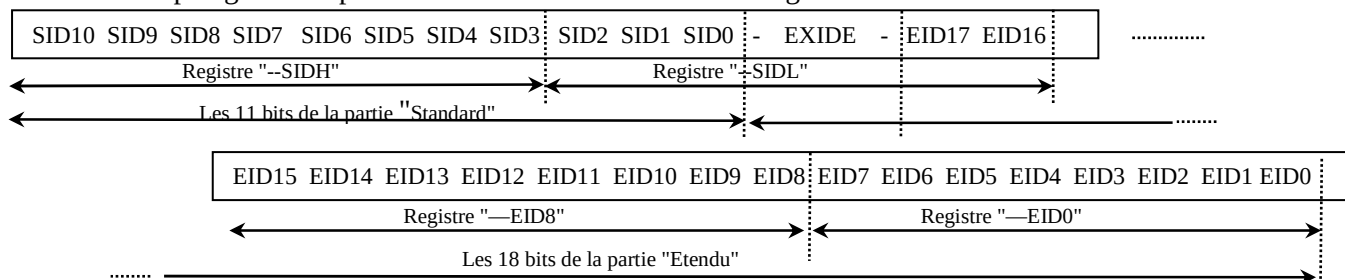
(d'après notice technique SJA1000 pages 40 et 41 MPC25050 pages 10 à 14)

Le repérage des différents bits dans les registres de configuration diffère selon qu'il s'agit du circuit SJA1000 (sur carte d'interface CAN PC104) ou du circuit MCP25050 qui équipe les différents modules d'interface CAN.

Repérage "SJA1000": 4 registres de 8 bits permettent de définir l'identificateur sur 29 bits (pour le 4^{ème} registre, les 3 bits LSB ne sont pas utilisés:



Repérage "MCP25050": de même 4 registre de 8 bits permettent de définir l'identificateur sur 29 bits mais le repérage et la répartition des bits dans les différents registres est différente:



Remarque: le Bit "EXIDE" est à mettre à 1 pour activer le mode étendu

Tableau des identificateurs pour nœuds CAN (Véhicule Multiplexé Didactique)

Se référer à la notice technique du circuit MCP25050, pages 10 à 14

Registre MCP25025 Et fonction	- SIDH (en bin)	- SIDL (en bin)	SIDH (Hex)	SIDL (Hex)	Identificateur (Hex) (! sur 29 bits)	Labels définis dans fichier fourni CAN_VMD.h
----------------------------------	--------------------	--------------------	---------------	---------------	---	--

Nœud "Asservissement"

RXF0 -> IRM et OM	000 0 0 1 00	001 - 1-xx	04	28	0084 xx xx	T_Ident_IRM_Asservissement
RXF1 -> IM	000 0 0 1 00	010 - 1-xx	04	48	0088 xx xx	T_Ident_IM_Asservissement
TXD0 -> On Bus	000 0 0 1 00	100 - 1-xx	04	88	0090 xx xx	
TXD1 -> Acq IM	000 0 0 1 01	000 - 1-xx	05	08	00A0 xx xx	T_Ident_AIM_Asservissement
TXD2 -> Mes. Auto.	000 0 0 1 10	000 - 1-xx	06	08	00C0 xx xx	

Nœud "Commodo feux"

RXF0 -> IRM et OM	001 0 1 0 00	001 - 1-xx	28	28	0504 xx xx	T_Ident_IRM_Commodo_Feux
RXF1 -> IM	001 0 1 0 00	010 - 1-xx	28	48	0508 xx xx	T_Ident_IM_Commodo_Feux
TXD0 -> On Bus	001 0 1 0 00	100 - 1-xx	28	88	0510 xx xx	
TXD1 -> Acq IM	001 0 1 0 01	000 - 1-xx	29	08	0520 xx xx	T_Ident_AIM_Commodo_Feux
TXD2 -> Mes. Auto.	001 0 1 0 10	000 - 1-xx	2A	08	0540 xx xx	

Nœud "Commodo essuie-glace"

RXF0 -> IRM et OM	001 0 1 1 00	001 - 1-xx	2C	28	0584 xx xx	T_Ident_IRM_Commodo_EG
RXF1 -> IM	001 0 1 1 00	010 - 1-xx	2C	48	0588 xx xx	T_Ident_IM_Commodo_EG
TXD0 -> On Bus	001 0 1 1 00	100 - 1-xx	2C	88	0590 xx xx	
TXD1 -> Acq IM	001 0 1 1 01	000 - 1-xx	2D	08	05A0 xx xx	T_Ident_AIM_Commodo_EG
TXD2 -> Mes. Auto.	001 0 1 1 10	000 - 1-xx	2E	08	05C0 xx xx	

Nœud "Feux avant gauche"

RXF0 -> IRM et OM	011 1 0 0 00	001 - 1-xx	70	28	0E04 xx xx	T_Ident_IRM_FVG
RXF1 -> IM	011 1 0 0 00	010 - 1-xx	70	48	0E08 xx xx	T_Ident_IM_FVG
TXD0 -> On Bus	011 1 0 0 00	100 - 1-xx	70	88	0E10 xx xx	
TXD1 -> Acq IM	011 1 0 0 01	000 - 1-xx	71	08	0E20 xx xx	T_Ident_AIM_FVG
TXD2 -> Mes. Auto.	011 1 0 0 10	000 - 1-xx	72	08	0E40 xx xx	

Nœud "Feux avant droit"

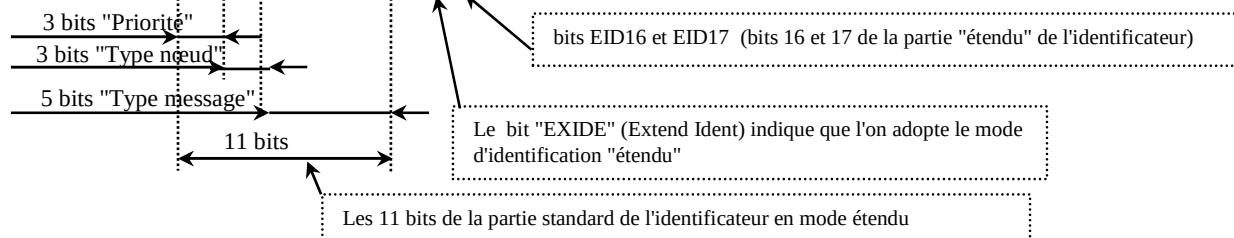
RXF0 -> IRM et OM	011 1 0 1 00	001 - 1-xx	74	28	0E84 xx xx	T_Ident_IRM_FVD
RXF1 -> IM	011 1 0 1 00	010 - 1-xx	74	48	0E88 xx xx	T_Ident_IM_FVD
TXD0 -> On Bus	011 1 0 1 00	100 - 1-xx	74	88	0E90 xx xx	
TXD1 -> Acq IM	011 1 0 1 01	000 - 1-xx	75	08	0EA0 xx xx	T_Ident_AIM_FVD
TXD2 -> Mes. Auto.	011 1 0 1 10	000 - 1-xx	76	08	0EC0 xx xx	

Nœud "Feux arrière gauche"

RXF0 -> IRM et OM	011 1 1 0 00	001 - 1-xx	78	28	0F04 xx xx	T_Ident_IRM_FRG
RXF1 -> IM	011 1 1 0 00	010 - 1-xx	78	48	0F08 xx xx	T_Ident_IM_FRG
TXD0 -> On Bus	011 1 1 0 00	100 - 1-xx	78	88	0F10 xx xx	
TXD1 -> Acq IM	011 1 1 0 01	000 - 1-xx	79	08	0F20 xx xx	T_Ident_AIM_FRG
TXD2 -> Mes. Auto.	011 1 1 0 10	000 - 1-xx	7A	08	0F40 xx xx	

Nœud "Feux arrière droit"

RXF0 -> IRM et OM	011 1 1 1 00	001 - 1-xx	7C	28	0F84 xx xx	T_Ident_IRM_FRD
RXF1 -> IM	011 1 1 1 00	010 - 1-xx	7C	48	0F88 xx xx	T_Ident_IM_FRD
TXD0 -> On Bus	011 1 1 1 00	100 - 1-xx	7C	88	0F90 xx xx	
TXD1 -> Acq IM	011 1 1 1 01	000 - 1-xx	7B	08	0FA0 xx xx	T_Ident_AIM_FRD
TXD2 -> Mes. Auto.	011 1 1 1 10	000 - 1-xx	7E	08	0FC0 xx xx	



Remarque:

Pour le circuit MCP25050 (notice technique MCP25050 pages 21), des 18 bits de la partie "étendu" de l'identificateur jouent un rôle particulier:

- les trois bits de poids faible définissent le type d'action, les EID8 à EID15 définissent dans certains cas, une adresse (lecture d'un registre).

4.2 Fichiers de définition fournis

Un certains nombre de fichiers de définition sont fournis:

- "CPU_Reg.h" définissant les adresses d'accès aux différents registres internes au microcontrôleur 68332 (processeur autour duquel est conçue la carte processeur EID210),
- "EID210_Reg.h" définissant les adresses d'accès aux différents éléments matériels implantés sur la carte processeur EID210,
- "Aton_CAN.h" définissant les adresses d'accès aux différents registres du contrôleur CAN SJA1000 implanté sur la carte industrielle "CAN-PC104" fournie par la société "ATON SYSTEMES",
- "Structures_Donnees.h" définissant l'ensemble des structures de données utiles à une programmation structurée,
- "CAN_VMD.h" définissant les éléments de l'application VMD (Véhicule Multiplexé didactique).



Veiller à ce que tous ces fichiers soient présents, soit dans le répertoire de travail, soit dans le répertoire "include" lui-même situé sous le répertoire contenant l'environnement de développement.

4.3 Structures de données spécifiques à l'application VMD

Pour utiliser le bus can, il faut utiliser le fichier de définition « aton_can.h ». Celui-ci définit les structures de données permettant la gestion des trames CAN.

L'union "ident_standard" permet de définir la partie identification en mode standard

```
typedef union
{
struct { // Eléments constitutif de l'identificateur dans une trame
    unsigned short ident:11; // les 11 bits d'identification en mode standard
    unsigned short nul:5; // 5 bits inutilisés
} identificateur;
struct { // Les mêmes éléments mais dans les registres du circuit SJA1000
    unsigned char ident1; // premier registre 8 bits de définition de l'identificateur
    unsigned char ident2; // deuxième registre 8 bits de définition de l'identificateur
} registre;
unsigned short valeur; // La taille globale est de 16 bits
} ident_standard;
```

L'union "ident_extend" permet de définir la partie identification en mode étendu

```
typedef union
{
struct { // Eléments constitutif de l'identificateur dans une trame
    unsigned long ident:29; // les 29 bits d'identification en mode étendu
    unsigned long x:3; // 3 bits inutilisés
} identificateur;
struct { // Les mêmes éléments mais dans les registres du circuit SJA1000
    unsigned char ident1; // premier registre 8 bits de définition de l'identificateur
    unsigned char ident2; // deuxième registre 8 bits de définition de l'identificateur
    unsigned char ident3; // troisième registre 8 bits de définition de l'identificateur
    unsigned char ident4; // quatrième registre 8 bits de définition de l'identificateur
} registre;
unsigned long valeur; // La taille globale est de 32 bits
} ident_extend;
```

La structure "Trame" permet de définir une trame complète

```
typedef struct
{
tr_info trame_info; // Taille 8 bits
union {
    ident_standard standard; // Identificateur en mode standard (2*8bits LSB)
    ident_extend extend; // Identificateur en mode étendu (4*8bits)
} ident;
unsigned char data[8]; // les 8 octets de données (au maximum)
} Trame; // Taille globale: 13 octets
```

Exemple: Définir une trame qui permette d'écrire dans un registre du circuit MCP25050 implanté sur le module 8 entrées "Commodo feux"

Dans ce cas, la trame envoyée par le contrôleur CAN (Circuit SJA1000 sur carte CAN_PC104) sera vue par le récepteur (circuit MCP25050 sur module) comme un "Input Message", avec la fonction "Write register" (voir documentation technique du MPC25025 pages 22).

Ce tableau indique que la trame comportera une zone de donnée composée de trois octets (donc DLC =3)

- le premier octet (repéré "addr") spécifie l'adresse du registre concerné par l'opération d'écriture
- le deuxième octet (repéré "mask") spécifie les bits concernés par l'opération d'écriture
- le troisième octet (repéré "value") spécifie la valeur des bits.

Remarques:

1- Rôle de l'octet "mask"

- Si on met un '1' dans un bit du masque, le bit correspondant du registre sera concerné par l'opération d'écriture. Il prendra la valeur spécifiée dans l'octet "value".
- Si on met un '0' dans un bit du masque, le bit correspondant du registre ne sera pas concerné par l'opération d'écriture. Il conservera sa valeur et la valeur spécifiée dans l'octet "value" est sans influence.

2- Détermination de l'adresse

Les adresses des différents registres sont définies dans la table 3 du chapitre 3, aux pages 15 et 16 de la notice technique du circuit MCP25050. Il faut prendre connaissance de la "note 1" qui indique qu'il faut ajouter 1Ch aux adresses indiquées dans le tableau, à l'exception du registre GPDDR (repère 1 en bas de la page 15)

Ex: pour écrire dans le registre de sortie "GPLAT", l'adresse sera 02h+1Ch = 1Eh

3- Détermination de l'identificateur

Dans l'annexe n°1 de ce document, il est décrit les principes d'identification des différents modules. Des labels sont également définis dans le registre CAN_VMD.H

→ Définition d'une variable structurée sous le modèle "Trame":

```
Trame T_IM_Commodo_Feux;           // IM -> Input Message
// Trame destinée au module 8 entrées sur lequel est connecté le commodo Essuie-Glace
```

Remarque: La variable structurée **T_IM_Commodo_Feux** comportera 8 octets (1 pour **trame_info**, 4 pour l'identificateur, 1 pour l'adresse registre, 1 pour le masque et 1 pour la valeur).

→ Accès et définition des différents éléments de la variable structurée "T_IM_Commodo_Feux"

```
T_IM_Commodo_Feux.trame_info.registre=0x00; // On initialise tous les bits du registre
à 0
T_IM_Commodo_Feux.trame_info.champ.extend=1; // On travaille en mode étendu
T_IM_Commodo_Feux.trame_info.champ.dlc=0x03; // Il y aura 3 données de 8 bits envoyés
T_IM_Commodo_Feux.trame_info.champ.rtr=0; // on veut une trame de donnée type IM
T_IM_Commodo_Feux.ident.extend.identificateur.ident= Ident_T_IM_Commodo_Feux; // voir
dans CAN_VMD.H
// Pour définir l'adresse du bloc optique arrière droit -> voir chapitre suivant
T_IM_Commodo_Feux.data[0]=0x1F; // "Adresse" -> registre concernée GPDDR
T_IM_Commodo_Feux.data[1]=0x7F; // "Masque" -> seuls les 7 bits de poids faibles seront affectés par
l'écriture
T_IM_Commodo_Feux.data[2]=0x7F; // "Valeur" -> on met à 1 les 7 bits de poids faibles
```

4.4 Bibliothèque de fonctions

Des fonctions spécifiques à l'accès au réseau CAN via la carte CAN-PC104 de chez "ATON SYSTEMES". Les prototypes de ces fonctions sont définis dans le fichier Aton_CAN.h et leurs définitions elle-mêmes dans le fichier "Aton_CAN.C"

S'il ne l'est déjà, ce fichier doit être compilé et son résultat de compilation Aton_CAN.o doit être inclus en tant que fichier à "Linker" avec le fichier application.



Veiller à ce que le fichier Aton_CAN.o soit présent dans le répertoire "lib" lui-même situé sous le répertoire contenant l'éditeur-assembleur EID210100.

Veiller également à ce que ce fichier Aton_CAN.o soit inclus au "linker".

Pour ce faire:

Dans logiciel EID210 [Configuration] puis [GNU C/C++] puis [Linker] puis [Ajouter] puis retrouver et sélectionner le fichier Aton_CAN.o qui viendra s'ajouter aux fichiers déjà présents c'est à dire CTR0.o et EID210.o

→ Fonction **Init_Aton_CAN()**

Cette fonction permet d'initialiser le circuit intégré SJA1000, cœur de la carte CAN-PC104 de chez ATON-SYSTEMES. Aucun paramètre n'est passé ni retourné.

Cette fonction est à appeler dans la partie "Initialisation" du programme application

→ Fonction **Ecrire_Trame(Nom_Trame)**

Cette fonction permet d'envoyer une trame sur le réseau CAN.

La variable passée est une variable structurée sous le modèle "Trame" (modèle défini dans le chapitre précédent).

Cette fonction ne renvoi pas de paramètre.

→ Fonction **Lire_Trame(&Nom_Trame_recue)**

Cette fonction permet de savoir si une trame a été reçue du réseau CAN et si oui d'accéder à celle-ci. La variable passée est un pointeur sur variable structurée sous le modèle "Trame" (modèle défini dans le chapitre précédent).

Le paramètre renvoyé vaut 1 si un message a été reçu et 0 si aucun message n'a été reçu.

→ Fonction **Affiche_Trame(Nom_Trame)**

Cette fonction permet d'afficher une trame sur l'écran de l'ordinateur connecté à l'unité centrale programmable.

La variable passée est une variable structurée sous le modèle "Trame" (modèle défini dans le chapitre précédent).

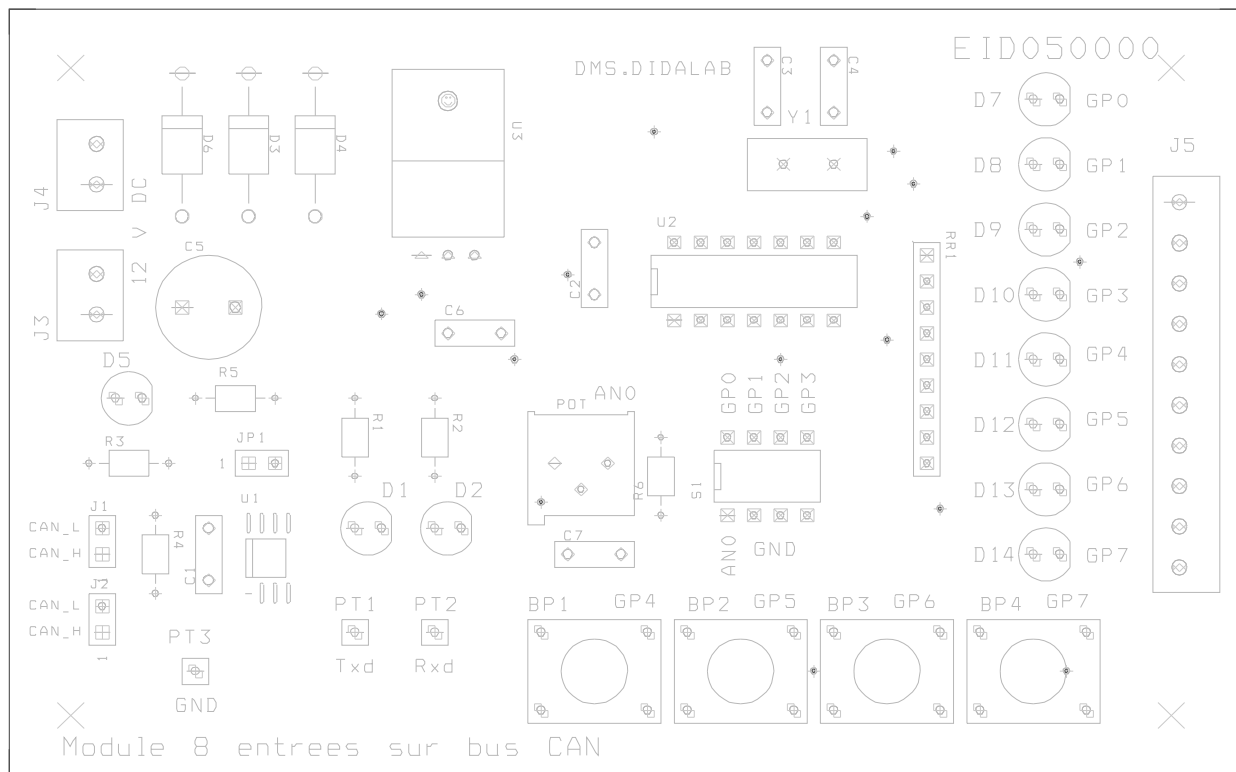
Cette fonction ne renvoi pas de paramètre.

5 ANNEXES

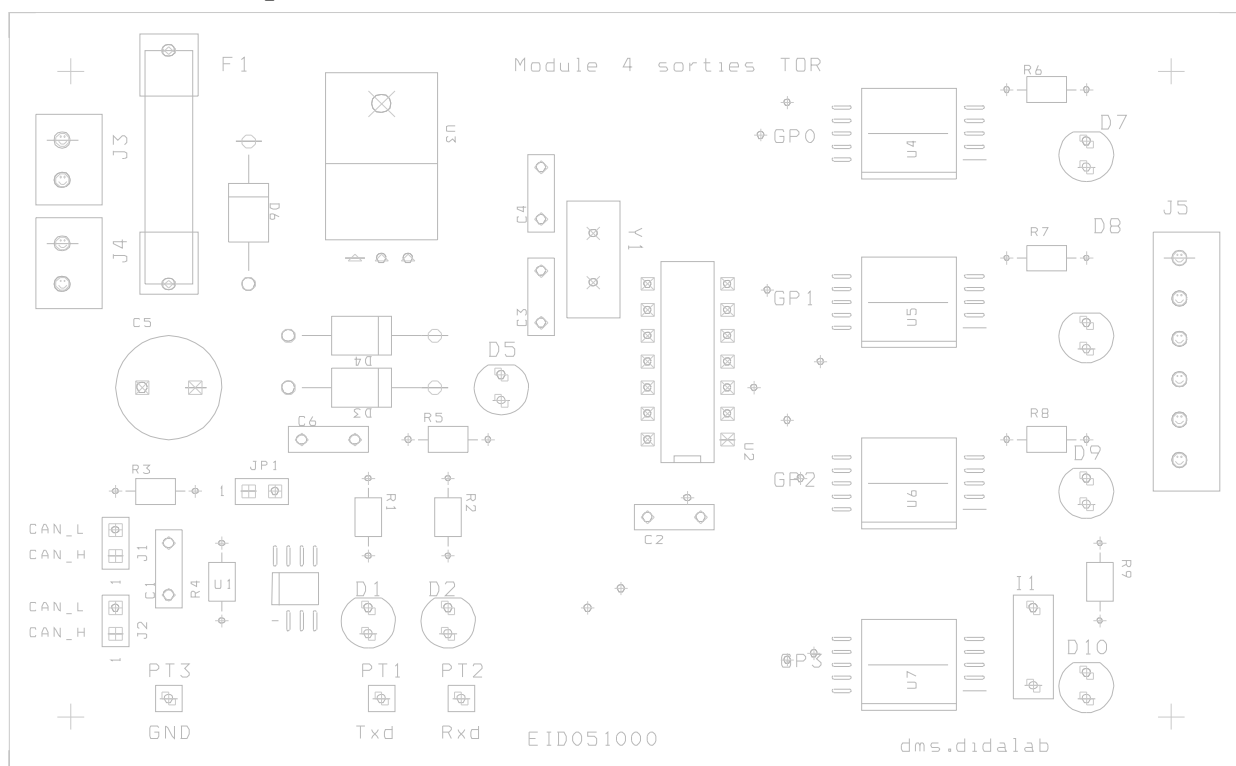
5.1 Cartes d'interface CAN

5.1.1 Plans d'implantation des composants

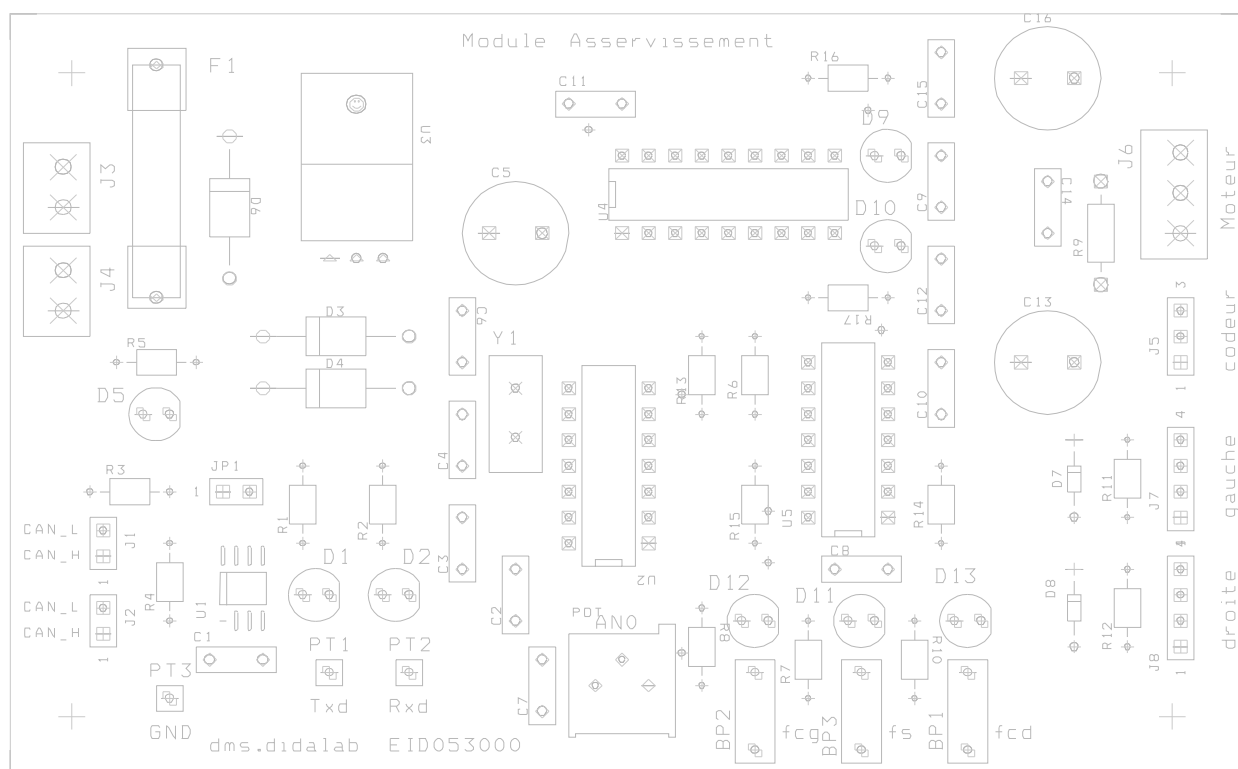
Carte 8 entrées / sorties logiques



Carte 4 sorties de puissance



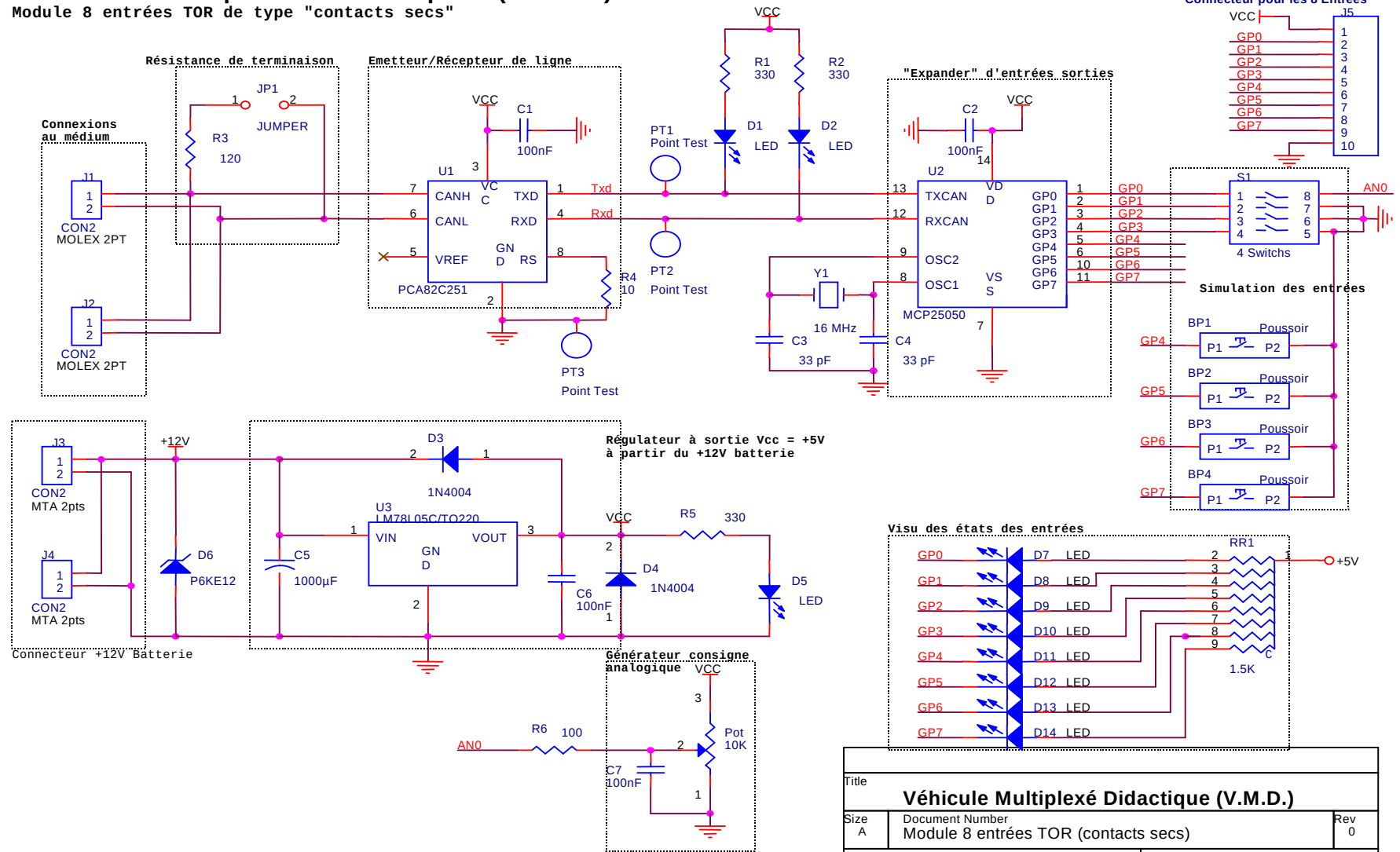
Carte "Asservissement"



5.1.2 Schémas structurels

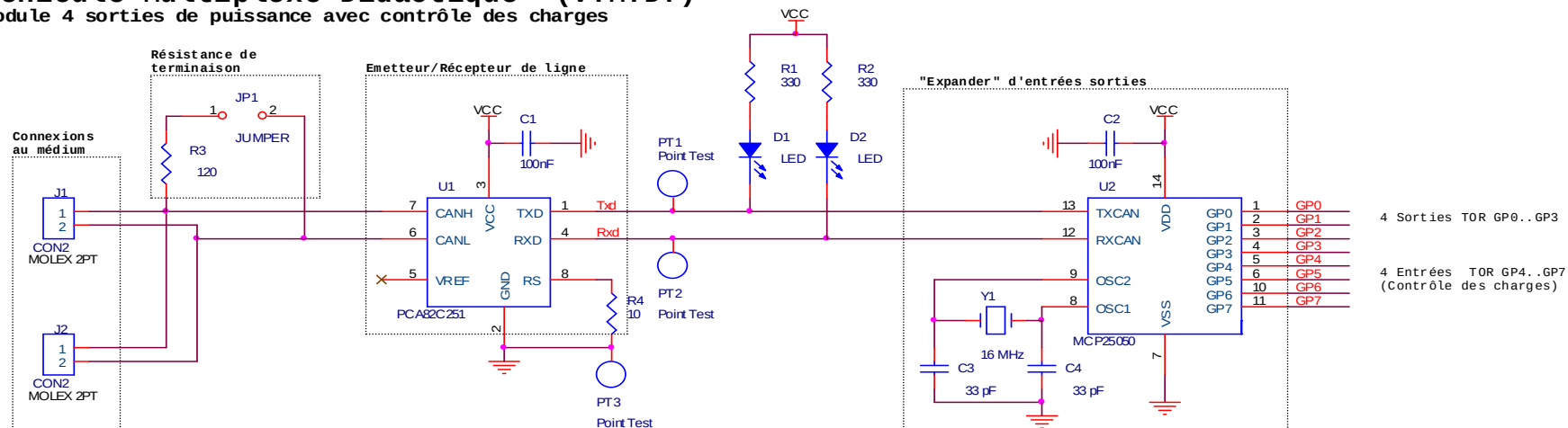
Véhicule Multiplexé Didactique (V.M.D.)

Module 8 entrées TOR de type "contacts secs"

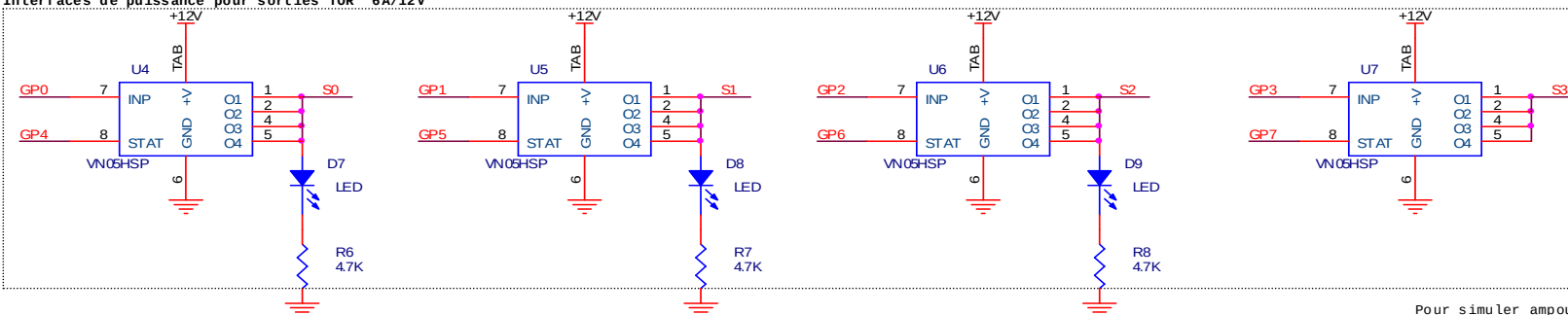


Véhicule Multiplexé Didactique (V.M.D.)

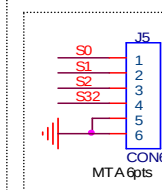
Module 4 sorties de puissance avec contrôle des charges



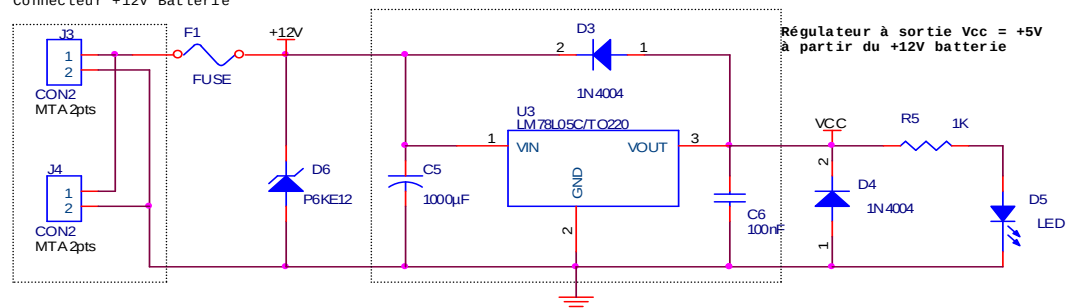
Interfaces de puissance pour sorties TOR 6A/12V



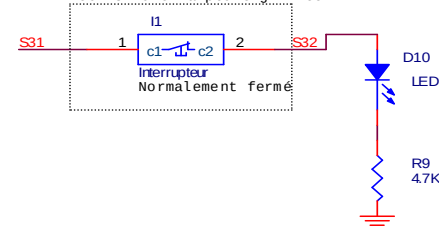
Connecteur charges électriques



Connecteur +12V Batterie



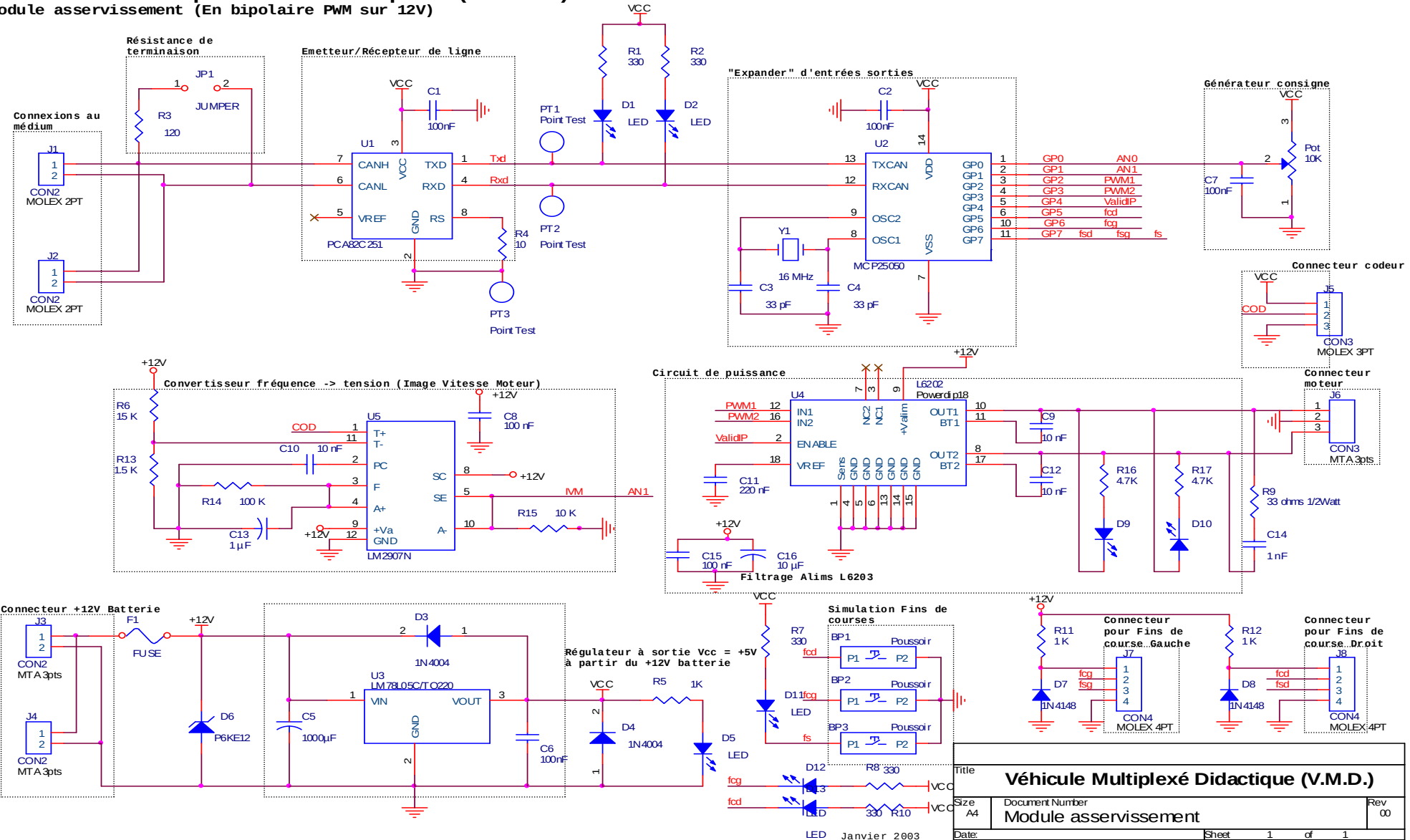
Pour simuler ampoule grillée



Title		
Véhicule Multiplexé Didactique (V.M.D.)		
Size A4	Document Number	Rev 00
Module 4 sorties TOR		
Date:	Sheet 1 of 1	

Janvier 2003

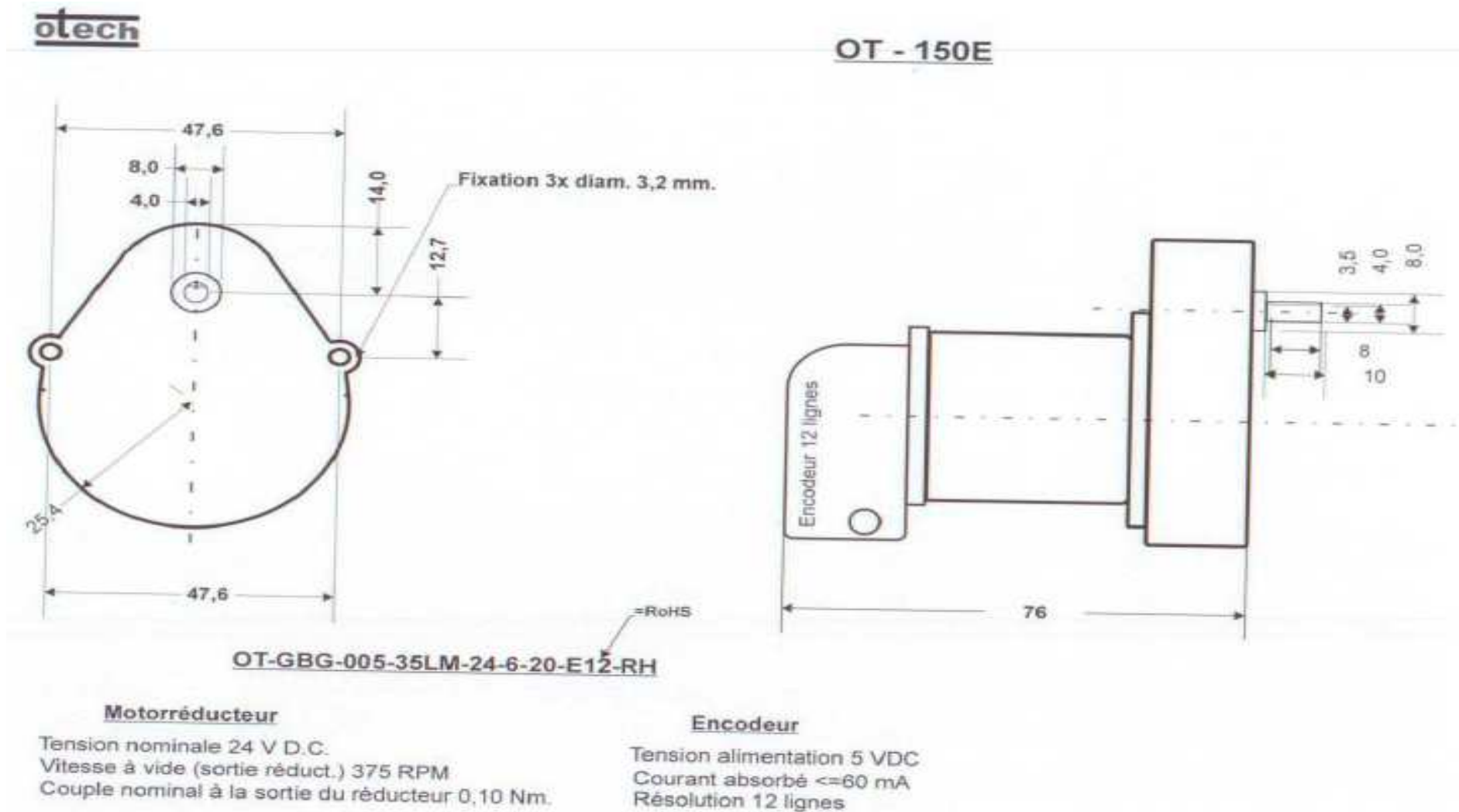
Véhicule Multiplexé Didactique (V.M.D.) Module asservissement (En bipolaire PWM sur 12V)



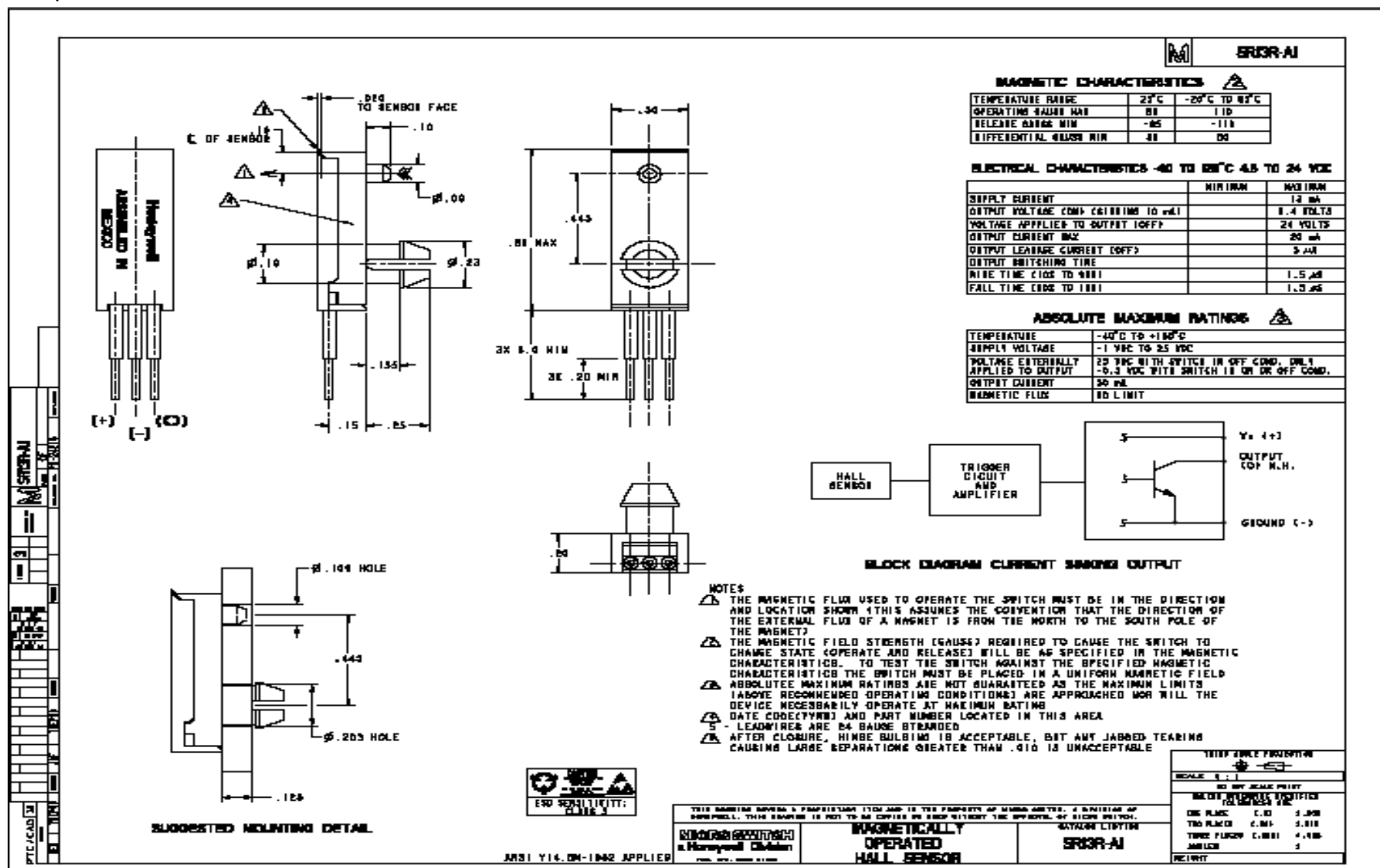
Title			Véhicule Multiplexé Didactique (V.M.D.)
Size			Document Number
A4			Module asservissement
Date:			Sheet 1 of 1
Date:			Rev 00

5.2 Éléments sur pare brise

5.2.1 Moto-réducteur-codeur



5.2.2 Capteurs de fins de courses



5.3 Fichier de définition CAN-VMD.h

```
/**
 * Structures de données pour application CAN VMD
 * Nom de fichier: CAN_VMD.h
 */

#ifndef _VMD_H
#define _VMD_H

// Format de message
typedef struct {
    /* nombre d'octets à envoyer, -1 si c'est une Remote Frame */
    int dlc;
    unsigned char id1; /* 8 bits de poids forts de l'ID. */
    unsigned char id2; /* 3 bits de poids faible de l'ID. */
    unsigned char data[8];
    } Message;

// Pour l'identificateur en mode standard
typedef union
{
    struct {
        unsigned short ident:11;
        unsigned short rtr:1;
        unsigned short nul:4;
    } identificateur;
    struct {
        unsigned char ident1;
        unsigned char ident2;
    } registre;
    unsigned short valeur;
} ident_standard;

// Pour l'identificateur en mode étendu
typedef union
{
    struct {
        unsigned long ident:29;
        unsigned long rtr:1;
        unsigned long x:2;
    } identificateur;
    struct {
        unsigned char ident1;
        unsigned char ident2;
        unsigned char ident3;
        unsigned char ident4;
    } registre;
    unsigned long valeur;
} ident_extend;

// Pour registre d'information Trame SJA1000
typedef union
{
    struct {
        unsigned char extend:1;
        unsigned char rtr:1;
        unsigned char nul:2;
        unsigned char dlc:4;
    } champ;
    unsigned char registre;
} tr_info;

// Pour Trame circulant sur réseau CAN
typedef struct
{
    tr_info trame_info;
    union {
        ident_standard standard;
        ident_extend extend;
    } ident;
    unsigned char data[8];
} Trame;
```

```
typedef union
{
struct
{
    unsigned char GP7:1;
    unsigned char GP6:1;
    unsigned char GP5:1;
    unsigned char GP4:1;
    unsigned char GP3:1;
    unsigned char GP2:1;
    unsigned char GP1:1;
    unsigned char GP0:1;
}bit;
unsigned char valeur;
}Port_8ES;
Port_8ES Etat_Commodo_Feux;
#define Valeur_Commodo_Feux Etat_Commodo_Feux.valeur
#define Cde_Veilleuse Etat_Commodo_Feux.bit.GP0
#define Cde_Warning Etat_Commodo_Feux.bit.GP1
#define Cde_Phare Etat_Commodo_Feux.bit.GP2
#define Cde_Code Etat_Commodo_Feux.bit.GP3
#define Cde_Clign_Gauche Etat_Commodo_Feux.bit.GP4
#define Cde_Clign_Droit Etat_Commodo_Feux.bit.GP5
#define Cde_Stop Etat_Commodo_Feux.bit.GP6
#define Cde_Klaxon Etat_Commodo_Feux.bit.GP7

// Pour la Commande des feux
#define Cde_Nulle 0x00
#define Cde_FV_V 0x01 // Feux aVant en Veilleuse
#define Cde_FR_V 0x01 // Feux aRrière en Veilleuse
#define Cde_FV_C 0x03 // Feux aVant en Code
#define Cde_FR_C 0x01 // Feux aRrière en Code
#define Cde_FV_P 0x05 // Feux aVant en Phare
#define Cde_FR_P 0x01 // Feux aRrière en Phare
#define Masque_Clign_AV 0x08 // Pour Clignotant AVant
#define Masque_Clign_AR 0x04 // Pour Clignotant ARrière
#define Masque_Klaxon 0x08 // Pour Claxon
#define Masque_Stop 0x02 // Pour Stop

// Variables pour la commande et le control des feux
//-----
// Pour variables images de la commande effectuée des différents feux
union byte_bits Image_FVG,Image_FVD,Image_FRD,Image_FRG;
// Pour Feux aVant Gauche
#define Valeur_FVG Image_FVG.valeur // Pour un accès port complet
#define Veilleuse_FVG Image_FVG.bit.b0
#define Code_FVG Image_FVG.bit.b1
#define Phare_FVG Image_FVG.bit.b2
#define Clignot_FVG Image_FVG.bit.b3
// Pour Feux aVant Droit
#define Valeur_FVD Image_FVD.valeur // Pour un accès port complet
#define Veilleuse_FVD Image_FVD.bit.b0
#define Code_FVD Image_FVD.bit.b1
#define Phare_FVD Image_FVD.bit.b2
#define Clignot_FVD Image_FVD.bit.b3
// Pour Feux aRrière Droit
#define Valeur_FRD Image_FRD.valeur // Pour un accès port complet
#define Veilleuse_FRD Image_FRD.bit.b0
#define Stop_FRD Image_FRD.bit.b1
#define Clignot_FRD Image_FRD.bit.b2
#define Klaxon_FRD Image_FRD.bit.b3
// Pour Feux aRrière Gauche
#define Valeur_FRG Image_FRG.valeur // Pour un accès port complet
#define Veilleuse_FRG Image_FRG.bit.b0
#define Stop_FRG Image_FRG.bit.b1
#define Clignot_FRG Image_FRG.bit.b2
#define Klaxon_FRG Image_FRG.bit.b3
// Pour les variables images des "Status" des sorties de puissance
union byte_bits Image_Stat_FVG,Image_Stat_FVD,Image_Stat_FRD,Image_Stat_FRG;
```

```
// Pour image "Status" Feux aVant Gauche
#define Valeur_Status_FVG Image_Stat_FVG.valeur // Pour un accès port complet
#define S_Veilleuse_FVG Image_Stat_FVG.bit.b4
#define S_Code_FVG Image_Stat_FVG.bit.b5
#define S_Phare_FVG Image_Stat_FVG.bit.b6
#define S_Clignot_FVG Image_Stat_FVG.bit.b7
// Pour image "Status" Feux aVant Droit
#define Valeur_Status_FVD Image_Stat_FVD.valeur // Pour un accès port complet
#define S_Veilleuse_FVD Image_Stat_FVD.bit.b4
#define S_Code_FVD Image_Stat_FVD.bit.b5
#define S_Phare_FVD Image_Stat_FVD.bit.b6
#define S_Clignot_FVD Image_Stat_FVD.bit.b7
// Pour image "Status" Feux aRrière Droit
#define Valeur_Status_FRD Image_Stat_FRD.valeur // Pour un accès port complet
#define S_Veilleuse_FRD Image_Stat_FRD.bit.b4
#define S_Stop_FRD Image_Stat_FRD.bit.b5
#define S_Clignot_FRD Image_Stat_FRD.bit.b6
#define S_Klaxon_FRD Image_Stat_FRD.bit.b7
// Pour image "Status" Feux aRrière Gauche
#define Valeur_Status_FRG Image_Stat_FRG.valeur // Pour un accès port complet
#define S_Veilleuse_FRG Image_Stat_FRG.bit.b4
#define S_Stop_FRG Image_Stat_FRG.bit.b5
#define S_Clignot_FRG Image_Stat_FRG.bit.b6
#define S_Klaxon_FRG Image_Stat_FRG.bit.b7

// Déclaration des identificateurs, pour les différents modules sur le bus VMD
//-----
// Pour Feux aVant Gauche
#define Ident_T_IRM_FVG 0x0E041E07 // Feux aVant Gauche en interrogation (Information Request Message)
#define Ident_T_IM_FVG 0x0E080000 // Feux aVant Gauche en commande (Input Message)
#define Ident_T_AIM_FVG 0x0E200000 // Feux aVant Gauche en Acquiescement commande
// Pour Feux aVant Droit
#define Ident_T_IRM_FVD 0x0E841E07 // Feux aVant Droit en interrogation (Information Request Message)
#define Ident_T_IM_FVD 0x0E880000 // Feux aVant Droit en commande (Input Message)
#define Ident_T_AIM_FVD 0x0EA00000 // Feux aVant Droit en Acquiescement commande
// Pour Feux aRrière Gauche
#define Ident_T_IRM_FRG 0x0F041E07 // Feux aRrière Gauche en interrogation (Information Request Message)
#define Ident_T_IM_FRG 0x0F080000 // Feux aRrière Gauche en commande (Input Message)
#define Ident_T_AIM_FRG 0x0F200000 // Feux aRrière Gauche en Acquiescement commande
// Pour Feux aRrière Droit
#define Ident_T_IRM_FRD 0x0F841E07 // Feux aRrière Droit en interrogation (Information Request Message)
#define Ident_T_IM_FRD 0x0F880000 // Feux aRrière Droit en commande (Input Message)
#define Ident_T_AIM_FRD 0x0FA00000 // Feux aRrière Droit en Acquiescement commande
// Pour Commodo Feux
#define Ident_T_IM_Commodo_Feux 0x05080000 // Commodo feux en commande (Information Message)
#define Ident_T_AIM_Commodo_Feux 0x05200000 // Commodo feux : Acquiescement suite à une IM
#define Ident_T_IRM_Commodo_Feux 0x05041E07 // Commodo feux en interrogation (Information Request Message)
// Pour Essuie Glace
#define Ident_T_IM_Commodo_EG 0x05880000 // Commodo EG en commande (Information Message)
#define Ident_T_AIM_Commodo_EG 0x05A00000 // Commodo EG en commande (Information Message)
#define Ident_T_IRM1_Commodo_EG 0x05841E07 // Commodo EG en interrogation: 1 octet demandé
#define Ident_T_IRM8_Commodo_EG 0x05840000 // Commodo EG en interrogation: 8 octets demandés
#define Ident_T_OB_Commodo_EG 0x05900000 // Commodo EG "On Bus" (Par scéduleur)
// Pour module Asservissement
#define Ident_T_IM_Asservissement 0x00880000 // "Asservissement" en commande (Input Message)
#define Ident_T_IRM1_Asservissement 0x00841E07 // "Asservissement" en interrogation: 1 octet en réponse
#define Ident_T_IRM8_Asservissement 0x00840000 // "Asservissement" en interrogation: 8 octets en réponse
#define Ident_T_AIM_Asservissement 0x00A00000 // "Asservissement" en Acquiescement commande
#define Ident_T_OB_Asservissement 0x00900000 // "Asservissement" en On Bus (par scéduleur)
// Pour le "Commodo Essuie Glace"
Port_8ES Commodo_EG;
#define Etat_Commodo_EG Commodo_EG.valeur
#define Valeur_Commodo_EG Commodo_EG.valeur
#define Cde_EG_Av_Int Commodo_EG.bit.GP0 // Commande Essuie Glace Avant en Intermittant
#define Entree1 Commodo_EG.bit.GP1
#define Entree2 Commodo_EG.bit.GP2
#define Cde_EG_Av_Pos1 Commodo_EG.bit.GP3 // Commande Essuie Glace Avant en Position1
#define Cde_EG_Av_Pos2 Commodo_EG.bit.GP4 // Commande Essuie Glace Avant en Position2
#define Cde_EG_Ar Commodo_EG.bit.GP5 // Commande Essuie Glace Arrière
#define Cde_Lave_Glace_Ar Commodo_EG.bit.GP6
#define Cde_Lave_Glace_Av Commodo_EG.bit.GP7
#endif
```

5.4 Fichier de définition propre à la carte ATON-CAN

```
// *****
// Fichier de définition pour carte controleur CAN "ATON_CAN"
// Nom de fichier: Aton_CAN.h
// *****

#ifndef _PELICAN_H
#define _PELICAN_H

#define SJA 0xB30280 /* Adresse de la carte SJA */
#define MODE *(unsigned char *) (SJA) /* Registre de controle */
#define COMMAND *(unsigned char *) (SJA+0x01) /* Registre de commande */
#define STATUS *(unsigned char *) (SJA+0x02) /* Registre status */
#define INTERRUPT *(unsigned char *) (SJA+0x03) /* Registre d'interruption */
#define INTERRUPT_ENABLE *(unsigned char *) (SJA+0x04)
#define BUS_TIMING_0 *(unsigned char *) (SJA+0x06) /* Registre bus timing 0 */
#define BUS_TIMING_1 *(unsigned char *) (SJA+0x07) /* Registre bus timing 1 */
#define OUTPUT_CONTROL *(unsigned char *) (SJA+0x08) /* Registre output control */
#define ARBITRATION_LOST_CAPTURE *(unsigned char *) (SJA+0x0B)
#define ERROR_CODE_CAPTURE *(unsigned char *) (SJA+0x0C)
#define ERROR_WARNING_LIMIT *(unsigned char *) (SJA+0x0D)
#define RX_ERROR_COUNTER *(unsigned char *) (SJA+0x0E)
#define TX_ERROR_COUNTER *(unsigned char *) (SJA+0x0F)
#define RX_FRAME_INFO *(unsigned char *) (SJA+0x10)
#define TX_FRAME_INFO *(unsigned char *) (SJA+0x10)

/* Mode SIMPLE FRAME */
#define RX_ID_1_S *(unsigned char *) (SJA+0x11)
#define RX_ID_2_S *(unsigned char *) (SJA+0x12)
#define RX_DATA_S (unsigned char *) (SJA+0x13) /* Adresse debut message */
#define TX_ID_1_S *(unsigned char *) (SJA+0x11)
#define TX_ID_2_S *(unsigned char *) (SJA+0x12)
#define TX_DATA_S (unsigned char *) (SJA+0x13) /* Adresse debut message */

/* Mode EXTENDED */
#define RX_ID_1_E *(unsigned char *) (SJA+0x11)
#define RX_ID_2_E *(unsigned char *) (SJA+0x12)
#define RX_ID_3_E *(unsigned char *) (SJA+0x13)
#define RX_ID_4_E *(unsigned char *) (SJA+0x14)
#define RX_DATA_E (unsigned char *) (SJA+0x15) /* Adresse debut message */

#define TX_ID_1_E *(unsigned char *) (SJA+0x11)
#define TX_ID_2_E *(unsigned char *) (SJA+0x12)
#define TX_ID_3_E *(unsigned char *) (SJA+0x13)
#define TX_ID_4_E *(unsigned char *) (SJA+0x14)
#define TX_DATA_E (unsigned char *) (SJA+0x15) /* Adresse debut message */

/* Les 2 modes */
#define RX_MESSAGE_COUNTER *(unsigned char *) (SJA+0x1D)
#define RX_BUFFER_START_ADDRESS *(unsigned char *) (SJA+0x1E)
#define CLOCK_DIVIDER *(unsigned char *) (SJA+0x1F)

/* acceptance code et mask en mode reset uniquement */
#define ACCEPT_CODE0 *(unsigned char *) (SJA+0x10)
#define ACCEPT_CODE1 *(unsigned char *) (SJA+0x11)
#define ACCEPT_CODE2 *(unsigned char *) (SJA+0x12)
#define ACCEPT_CODE3 *(unsigned char *) (SJA+0x13)

#define ACCEPT_MASK0 *(unsigned char *) (SJA+0x14)
#define ACCEPT_MASK1 *(unsigned char *) (SJA+0x15)
#define ACCEPT_MASK2 *(unsigned char *) (SJA+0x16)
#define ACCEPT_MASK3 *(unsigned char *) (SJA+0x17)

/** Configurations des bits des registres */
/* Registre STATUS */
#define BUS_STATUS 0x80
#define ERROR_STATUS 0x40
#define TRANSMIT_STATUS 0x20
#define RECEIVE_STATUS 0x10
#define TRANSMISSION_COMPLETE 0x08
#define TRANSMIT_BUFFER_STATUS 0x04
#define DATA_OVERRUN_STATUS 0x02
#define RECEIVE_BUFFER_STATUS 0x01
```

```
// ***** TYPES *****
// *****
// *
// *   DECLARATIONS
// *   des prototypes des fonctions spécifiques CAN - SJA1000
// *****
/** Initialisation du SJA1000 en mode PeliCAN. */
void init_sja1000_peli_acc (char acc_code0, char acc_code1, char acc_code2, char acc_code3,
                           char acc_mask0, char acc_mask1, char acc_mask2, char acc_mask3);
/** Initialisation du SJA1000 en mode PeliCAN. */
void init_sja1000_peli ();
Trame Receive_Trame();
/** Envoi d'une trame */
void send_trame_peli (Message mes);
/** Reception d'un message. */
Message receive_trame_peli ();
/** Affichage des regsitres en PeliCAN */
void print_reg_peli ();
/* Affiche l'etat du STATUS REGISTER */
void show_status ();
/* Affiche le message passe en parametres sur une ligne */
void print_little (Message mes);
// initialisation de la carte CAN ATON
void Init_Aton_CAN();
char Ecrire_Trame(Trame message);
char Lire_Trame(Trame *message_recu);
void Affiche_Trame(Trame trame);
```